Foundations of Data Science

Machine Learning

In computer science, there are many tasks that are too complex to hand-code (such as object recognition, natural language processing, and speech recognition). To overcome this limitation, it was invented the **Machine Learning (ML).** The idea is to build algorithms that learn automatically from data or experience.

Depending on the **nature of the data** and the **feedback signal** provided to the learning agent, machine learning is traditionally categorized into three fundamental paradigms

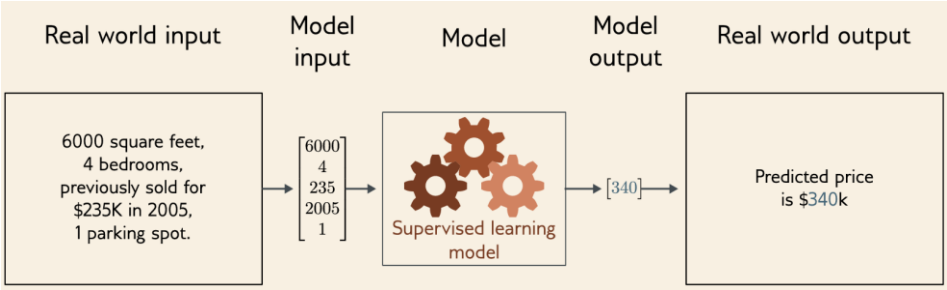
Supervised Learning

In traditional computing, an engineer provides the data and the program to the computer to calculate the output. In Supervised Learning, instead, we give to the computer pairs of input-output to discover the underlying program (mapping function).

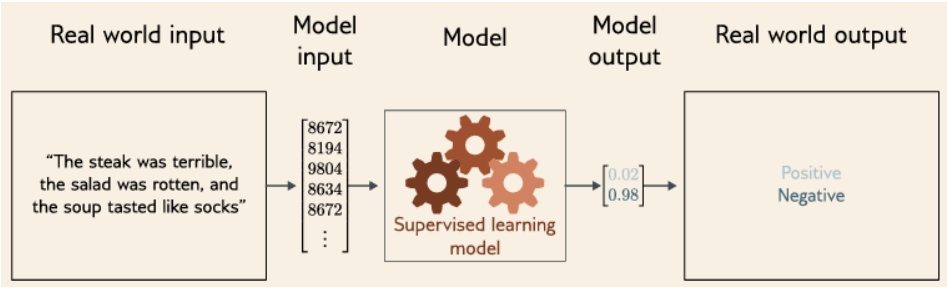
- **Input**: Examples of inputs and desired outputs
- **Output**: Model that predicts output given a new input

Supervised learning problems are grouped by the nature of the target variable $\boldsymbol{y}$:

1. **Regression Tasks:** The output target $\boldsymbol{y}$ is continuous (e.g., predicting house prices, temperature, or coordinate trajectories).



2. **Classification Tasks:** The output target $\boldsymbol{y}$ is discrete and categorical. We can also have different types of classification.
    - *Binary Classification:* The target belongs to one of two mutually exclusive classes.
    - *Multiclass Classification:* The target belongs to one of more than two discrete classes (e.g., classifying a text document into distinct topic categories, or classifying audio signals into music genres).



Self-Supervised Learning: However, Supervised tasks frequently suffer from label scarcity because acquiring human annotations is expensive. So, modern ML bridges this gap by learning data structures from massive unlabeled datasets first (e.g., predicting masked words in text). Once the model internalizes the structural rules of the input space, adapting it to a supervised downstream task requires significantly fewer labeled training pairs.

Unsupervised Learning

In this type of ML, instead, **we do not provide labelled pairs**. We only feed the algorithm with examples of data. The goal is finding "interesting" patterns in the data

- **Input**: Examples of some data (no "outputs")
- **Output**: Representation of structure in the data

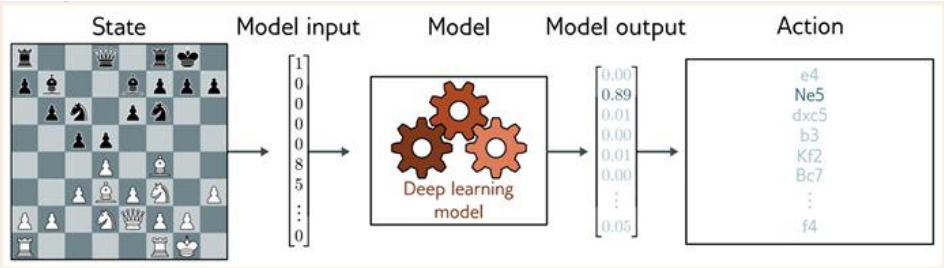
Key applications and model families include:

- **Clustering:** Partitioning data into homogeneous subsets based on similarity metrics (e.g., grouping customer purchasing behaviors).
- **Outlier & Anomaly Detection:** Identifying samples that deviate substantially from the nominal data distribution.
- **Data Imputation:** Inferring and filling in missing values within incomplete records.
- **Generative Modeling:** Learning the underlying probability distribution $\boldsymbol{p(x)}$ over the data space to synthesize realistic new samples.

Reinforcement learning

Reinforcement Learning designs active learning systems (agents) that interacts directly with an external environment over discrete time steps to maximize a cumulative numerical reward. Unlike supervised learning, no dataset of correct actions is provided; the agent must generate its own data by exploring the environment.

- **Input**: Sequence of interactions with an environment
- **Output**: Policy that performs a desired task



Reinforcement learning is uniquely difficult due to three fundamental issues:

1. **Stochasticity:** Real-world transitions and reward outcomes are frequently non-deterministic. Selecting the exact same action from the same state may yield different consequences depending on environment dynamics or an opponent's unpredictable choices.
2. **Temporal Credit Assignment Problem:** Rewards are often sparse and significantly delayed. An agent winning a game after dozens of moves must determine which specific decisions earlier in the trajectory were responsible for the eventual victory versus which were suboptimal.
3. **Exploration vs. Exploitation Trade-off:** The agent must continuously balance:
    - *Exploitation:* Executing known actions that currently yield reliable positive rewards.
    - *Exploration:* Trying unfamiliar actions to discover potentially superior strategies that could yield higher long-term cumulative gain.

Introduction to Supervised Learning

Def. Supervised learning fundamentally relies on structured observations taken from $\boldsymbol{n}$ distinct entities or objects.

Each observation is divided into two primary components:

- **Features (inputs):** represent measurable properties or attributes that describe an object
- **Target (output):** represent the outcome or variable that an algorithm aims to predict

The global objective of supervised machine learning is to uncover the underlying predictive rule or mathematical mapping that links the feature values to their corresponding target values.

Example: A classic illustration of this framework is the Iris Dataset. Commonly used as a foundational benchmark or "toy example," it contains 150 instances of iris flowers categorized equally across three subspecies.

- Features: sepal length, sepal width, petal length, and petal width, all recorded in centimeters.
- Target: the categorical flower species to which each observation belongs.

Variable Types and Problem Formulation

Both features and target may be of different data types, each requiring distinct mathematical treatment

- **Numerical variables:** → **R**
- **Integer variables:** → **Z**
- **Categorical variables:** → **{C1, ..., Cg }**
- **Binary variables:** → **{0, 1}**

The nature of the target variable directly dictates the primary task of supervised learning:

- **Regression:** Occurs when the target space is continuous (numerical, $\mathbb{R}$).
- **Classification:** Occurs when the target space is discrete or categorical ($\{\boldsymbol{C_1}, \dots, \boldsymbol{C_g}\}$).

A dataset whose target values are known and recorded is referred to as **labeled data**. Conversely, when the target values are absent or unobserved, the dataset is defined as **unlabeled data**

Encoding for Categoricals

Most machine learning algorithms are designed to operate **exclusively over numerical** data because their underlying mechanisms rely on vector algebra, distance metrics, and matrix computations. Therefore, **non-numeric features must be transformed into a numerical** format through an appropriate **encoding** strategy.

When dealing with nominal categorical variables containing with **k mutually exclusive categories** is transformed from a scalar value into a k-dimensional binary vector. This vector has exactly one element set to 1 to indicate the category, and 0s elsewhere:

$$\mathbf{e}_i = (0, \dots, 0, \underbrace{1}_{i\text{-th position}}, 0, \dots, 0)^\top, \quad i \in \{1, \dots, k\}$$

If a categorical variable exhibits a natural progression or ranking (an ordinal variable, such as rating tiers), the encoding must preserve this hierarchical relationship. In such cases, categories are sequentially mapped to an ordered scale, such as a sequence of integers.

Dataset

Def. **A dataset is an organized collection of related data points** grouped together for analysis, processing, or machine learning

Mathematically we define a dataset D as a set of couples input -> output, or, features -> target:

$$\boldsymbol{D} = \{(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)}), \dots, (\boldsymbol{x}^{(n)}, \boldsymbol{y}^{(n)})\} \subset (\boldsymbol{X}\ \boldsymbol{x}\ \boldsymbol{Y})^{\boldsymbol{n}}$$

Where:

- X is the input/feature space, typically $X \subseteq \mathbb{R}^{\mathrm{P}}$ (where P is the number of features)
- Y is the output/target space

Each observation (row) corresponds to a tuple:

$$\boldsymbol{row} = (\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)}) \subset \boldsymbol{X}\ \boldsymbol{x}\ \boldsymbol{Y} \qquad \boldsymbol{i} = \boldsymbol{1}, \dots, \boldsymbol{n}$$

The j-th feature across all $\boldsymbol{n}$ samples can be isolated as a column vector:

$$\boldsymbol{x_j} = (\boldsymbol{x_j}^{(1)}, \dots, \boldsymbol{x_j}^{(n)}) \in \boldsymbol{R}^{\boldsymbol{n}}$$

Data Generation Process

In statistical machine learning, the dataset $\mathcal{D}$ is assumed to originate from a real-world probabilistic mechanism known as the **data generation process**. This process is governed by a joint probability distribution:

$$\boldsymbol{P_{XY}}\ \textbf{defined over the product space}\ \boldsymbol{\mathcal{X} \times \mathcal{Y}}.$$

In this context, features and targets are treated as random variables, conventionally denoted by lowercase $\boldsymbol{x}$ and $\boldsymbol{y}$. However, **the true distribution PXY is usually unknown**. So, the primary objective of supervised learning is to deduce its structure from observed data to accurately infer $\boldsymbol{y}$ given a novel instance $\boldsymbol{x}$.

I.I.D. Assumption

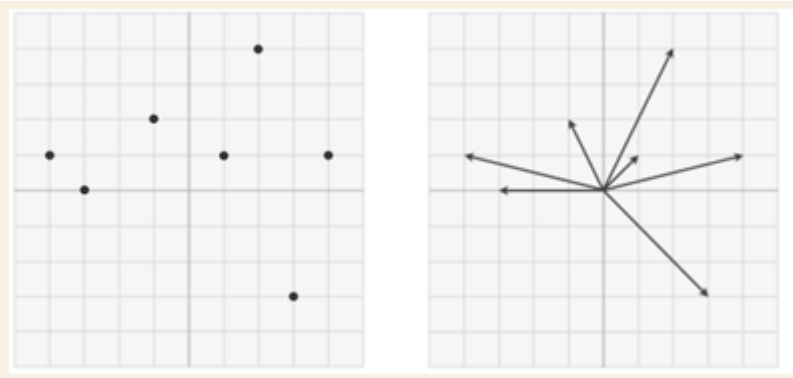
A standard foundational premise in supervised learning is the **i.i.d. assumption** (independent and identically distributed):

1. **Identically Distributed:** Every observed pair $(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)})$ is sampled from the exact same underlying distribution $\boldsymbol{P_{XY}}$.
2. **Mutually Independent:** The realization of the $\boldsymbol{i}$-th sample does not influence, nor is it influenced by, the remaining $\boldsymbol{n - 1}$ samples.

While the i.i.d. assumption simplifies statistical learning theory, it remains an idealization that does not always hold in complex operational domains

Data Representation

Restricting attention to the input space $\mathcal{X}$, every sample x_p can be conceptualized geometrically as a point or vector in a p-dimensional real coordinate space.



Mean Centering

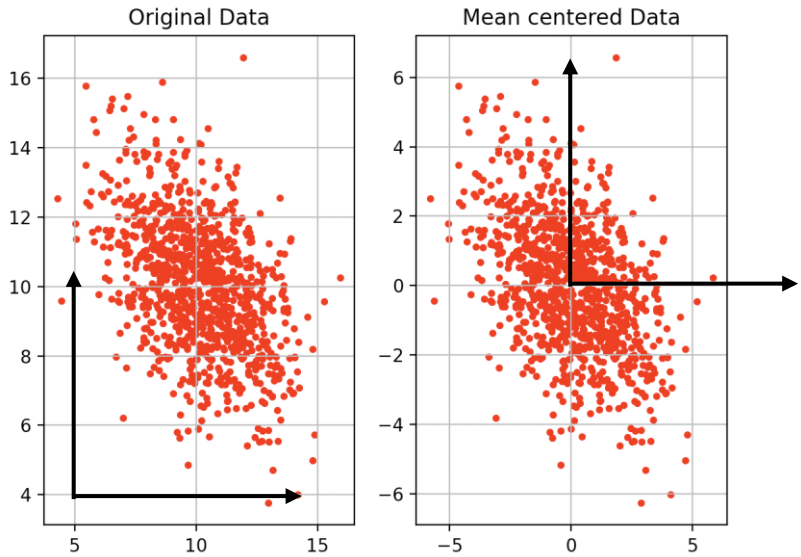
A common geometric preprocessing step is **mean-centering**. In this operation, the empirical mean of each feature dimension is subtracted from every observation:

$$\tilde{x}^{(i)} = x^{(i)} - \overline{x}$$

Where:

$$\overline{x} = \frac{1}{n}\sum_{i=1}^{n} x^{(i)}.$$

Mean-centering shifts the entire point cloud such that its center of mass sits directly on the origin, while leaving the internal geometric relationships between observations intact. Because it is a simple translation of coordinates, the operation is completely reversible.



Spanning Set

Def. A spanning set is a collection of vectors that can be combined (using addition and scalar multiplication) to produce every vector in a space.

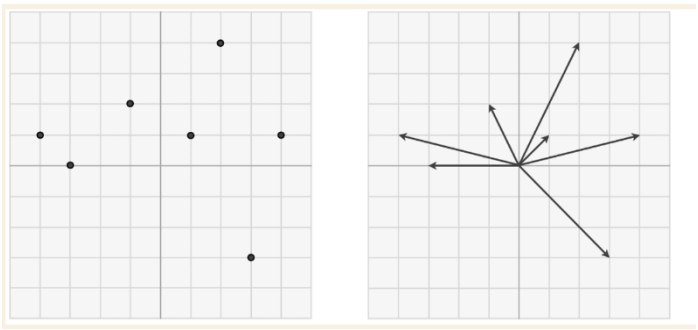
Example: In 2D space, the two vectors ***e1 = [1 0]*** and ***e2 = [0 1]*** span the entire plane, because any point ***k*** can be written as a linear combination of e1 and e2:

$$k = x(e1) + y(e2) = [x\ y]$$

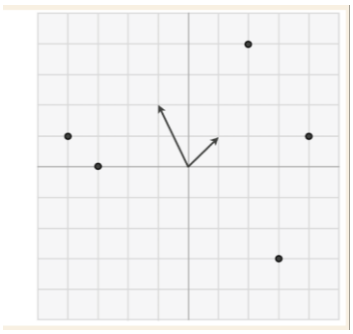
Basis

Def. **A basis is a minimal spanning set**, meaning that it is a **set of vectors** that preserves two fundamental characteristics:

- **Spanning set (Generators):** Any vector in the space can be written as a linear combination of these vectors.
- **Linearly Independent:** No vector in the set can be written as a combination of the others. If this happens, we can say that the vector is "redundant" and must be eliminated from the basis set.



Each data point can be represented as a dot or an arrow from the origin.



Instead of keeping all vectors, we can choose a minimal set as a Basis. All other data points can be uniquely reconstructed from these base vectors.

Example: In 2D space, the set:

$$C = \{e1, e2\} = \{[1\ 0], [0\ 1]\}$$

is a basis, because it spans the entire space R^2 AND the two vectors are linearly independent (neither is a multiple of the other).

Golden Rule

If we have a N-dimensional space and we have exactly N vectors (e.g. R^2 and 2 vectors, R^3 and 3 vectors, …, R^N and N vectors), then:

Linearly Independent <=> Spanning Set

This means that with exactly N vectors in an N-dimensional space, we do NOT need to check both properties! If they are linearly independent, they automatically span the space (and form a basis). If they span the space, they are automatically linearly independent (and form a basis).

How does this apply to Data Science

If we manage to find a basis $\boldsymbol{C} = \{\boldsymbol{c_1}, \dots, \boldsymbol{c_N}\}$ then for any point $\boldsymbol{x_p}$ in our dataset we will be able to find a set of numbers $\boldsymbol{w_{1,p}}, \dots, \boldsymbol{w_{N,p}}$ such that:

$$\boldsymbol{x_p} = \boldsymbol{w_{1,p}}\,\boldsymbol{c_1} + \ \dots + \boldsymbol{w_{N,p}}\,\boldsymbol{c_N}$$

(this means to find the linear combination of the basis that generates x_p)

The symbols mean:

- $\boldsymbol{N}$**:** is the dimension of the space (e.g., $\boldsymbol{N} = \boldsymbol{2}$ for two-dimensional points).
- $\boldsymbol{P}$**:** is the total number of data points (also named samples).
- $\boldsymbol{x_p}$**:** is the $\boldsymbol{p}$-th individual data point (expressed as a vector of N numerical values/features).
- $\boldsymbol{c_1}, \boldsymbol{c_2}, \dots, \boldsymbol{c_N}$**: are the $\boldsymbol{N}$ vectors chosen as the basis;** *(each $\boldsymbol{c_i}$ is expressed as a vector of N numerical values).*
- $\boldsymbol{w_{i,p}}$**:** is the weight assigned to the *i-th* basis ($\boldsymbol{c_i}$) to reconstruct the point $\boldsymbol{x_p}$

Standard Basis and Orthonormality

Each standard basis (or canonical basis) vector consists of zeros everywhere except for a 1 in the k-th position:

$$\boldsymbol{c_k} = [\boldsymbol{0}, \boldsymbol{0}, \dots, \boldsymbol{1}, \dots, \boldsymbol{0}]$$

Each standard basis preserves 2 key properties*:*

- **Orthogonality**: the dot product of two different vectors of the basis is always null.

$$\boldsymbol{c_n^T c_m} = \boldsymbol{0} \quad (\text{for } \boldsymbol{n} \neq \boldsymbol{m})$$

- **Normalization**: Every basis vector has a length of exactly 1, meaning that the vectors are at a distance of exactly 1 from the origin (0, 0)

$$\|\boldsymbol{c_n}\|_2^2 = \boldsymbol{1} \quad \text{for } \boldsymbol{n} = \boldsymbol{1} \dots \boldsymbol{N}$$

Together, these two properties make the set ORTHONORMAL. Note that the same constraints expressed in terms of the concatenated basis matrix C can be written compactly as:

$$\boldsymbol{C^T C} = \boldsymbol{I_{N \times N}}$$

EXPLAINATION: Invece di considerare i vettori della base ortonormale $\boldsymbol{c_1}, \boldsymbol{c_2}, \dots, \boldsymbol{c_N}$ separati, li possiamo affiancare come colonne di una sola matrice quadrata $\boldsymbol{N} \times \boldsymbol{N}$ (dove N è la dimensione dello spazio) ed otteniamo:

$$\boldsymbol{C} = \begin{bmatrix} | & | & & | \\ \boldsymbol{c_1} & \boldsymbol{c_2} & \dots & \boldsymbol{c_N} \\ | & | & & | \end{bmatrix}$$

(nota: stiamo considerando una qualsiasi base ortonormale, non per forza una base standard / canonica di tutti 0 e 1 sulla diagonale!).

A questo punto, quando facciamo il prodotto tra la matrice ($\boldsymbol{C}$) e la sua trasposta ($\boldsymbol{C^T}$) in cui scambiamo righe e colonne, otteniamo:

$$\boldsymbol{C^T C} = \begin{bmatrix} - & \boldsymbol{c_1^T} & - \\ - & \boldsymbol{c_2^T} & - \\ & \vdots & \\ - & \boldsymbol{c_N^T} & - \end{bmatrix} \begin{bmatrix} | & | & & | \\ \boldsymbol{c_1} & \boldsymbol{c_2} & \dots & \boldsymbol{c_N} \\ | & | & & | \end{bmatrix} = \begin{bmatrix} \boldsymbol{c_1^T c_1} & \boldsymbol{c_1^T c_2} & \dots & \boldsymbol{c_1^T c_N} \\ \boldsymbol{c_2^T c_1} & \boldsymbol{c_2^T c_2} & \dots & \boldsymbol{c_2^T c_N} \\ \vdots & \vdots & \ddots & \vdots \\ \boldsymbol{c_N^T c_1} & \boldsymbol{c_N^T c_2} & \dots & \boldsymbol{c_N^T c_N} \end{bmatrix}$$

MA:

- **Sulla diagonale principale** ci sono solo prodotti di un vettore con sé stesso: $\boldsymbol{c_n^T c_n} = \|\boldsymbol{c_n}\|_2^2$. Dato che i vettori sono normalizzati a lunghezza $\mathbf{1}$, questi valori valgono tutti $\mathbf{1}$.
- **Fuori dalla diagonale** ci sono solo prodotti tra vettori diversi: $\boldsymbol{c_n^T c_m}$ con $\boldsymbol{n \neq m}$, che, grazie alla proprietà di ortogonalità vista prima, danno sempre come risultato $\mathbf{0}$.

Dunque il risultato finale è:

$$\boldsymbol{C^T C} = \begin{bmatrix} \mathbf{1} & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{1} & \dots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \dots & \mathbf{1} \end{bmatrix} = \boldsymbol{I_{NxN}}$$

Cioè la matrice identità, in cui abbiamo 0 in tutte le posizioni ed 1 sulla diagonale.

Encoding

Nelle slide precedenti avevamo visto che ricostruire un punto $\boldsymbol{x_p}$ a partire da una base $\boldsymbol{C}$ significa scrivere una combinazione lineare:

$$\boldsymbol{x_p = c_1 w_{1,p} + c_2 w_{2,p} + \dots + c_N w_{N,p}}$$

Example: Assume we are using a standard basis:

$$\boldsymbol{x_p} = [\mathbf{3.5}, \mathbf{-2.1}, \mathbf{7.0}] \qquad \boldsymbol{x_p} = \mathbf{3.5} \cdot \begin{bmatrix} \mathbf{1} \\ \mathbf{0} \\ \mathbf{0} \end{bmatrix} + (\mathbf{-2.1}) \cdot \begin{bmatrix} \mathbf{0} \\ \mathbf{1} \\ \mathbf{0} \end{bmatrix} + \mathbf{7.0} \cdot \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \mathbf{1} \end{bmatrix}$$

$w_{1,p}$ $w_{2,p}$ $w_{3,p}$

Representing a data point $\mathbf{x_p}$ over the standard basis is very easy, since each weight is defined as $\mathbf{w_p = x_p}$.

Se compattiamo i vettori della base nella matrice $\boldsymbol{C} = [\boldsymbol{c_1}, \dots, \boldsymbol{c_N}]$ e i pesi nel vettore $\boldsymbol{w_p} = [\boldsymbol{w_{1,p}}, \dots, \boldsymbol{w_{N,p}}]$, questa uguaglianza diventa semplicemente una moltiplicazione matrice-vettore:

$$\boldsymbol{C w_p = x_p}$$

MA siccome abbiamo detto che noi lavoreremo con basi ortonormali AND $\boldsymbol{C^T C = I}$ allora possiamo moltiplicare da entrambi i lati per $\boldsymbol{C^T}$ ed ottenere:

$$\cancel{\boldsymbol{C^T C}}\, \boldsymbol{w_p} = \boldsymbol{C^T x_p}$$

$$\boldsymbol{w_p = C^T x_p}$$

A questo punto non dobbiamo più risolvere alcun sistema di equazioni poiché ci basta fare un semplice prodotto matrice-vettore. Questo processo, in Data Science, si chiama Encoding.

Def. In Data Science e Machine Learning, chiamiamo $\boldsymbol{w_p}$ l'**encoding** (la codifica o rappresentazione interna) del data point originale $\boldsymbol{x_p}$.

Non-Standard Basis

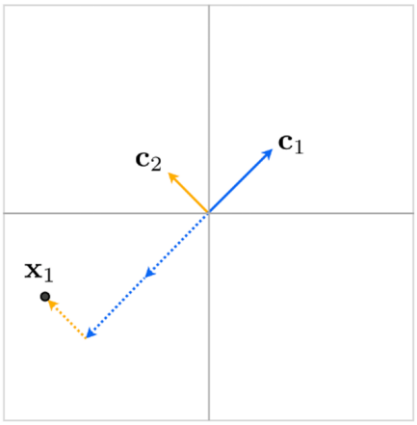
Let's start with an example. Let's choose a basis that's not the standard one. For example:

$$\boldsymbol{c_1} = \begin{bmatrix}\mathbf{2}\\ \mathbf{1}\end{bmatrix}, \quad \boldsymbol{c_2} = \begin{bmatrix}\mathbf{1}\\ \mathbf{2}\end{bmatrix}$$

Any generic data point $\boldsymbol{x_p}$ can therefore be represented as a unique linear combination:

$$\boldsymbol{x_p} = \boldsymbol{w_{1,p}c_1} + \boldsymbol{w_{2,p}c_2}$$

Let's take, for example, the point $\boldsymbol{x_1} = (\mathbf{-3}, \mathbf{0})$. With the standard basis, to reach the point $\boldsymbol{x_1}$, we would move 3 steps left and 0 steps up/down. But, what happens in the new plane generate from the new basis $\boldsymbol{C} = \{\boldsymbol{c_1}, \boldsymbol{c_2}\} = \{\begin{bmatrix}\mathbf{2}\\ \mathbf{1}\end{bmatrix}, \begin{bmatrix}\mathbf{1}\\ \mathbf{2}\end{bmatrix}\}$?



To reach $\boldsymbol{x_1}$ using our new basis C = $\{\boldsymbol{c_1}, \boldsymbol{c_2}\}$, we walk -2 steps along vector $\boldsymbol{c_1}$ (shown as the blue dotted line pointing in the opposite direction) and +1 step along vector $\boldsymbol{c_2}$ (the orange dotted line). This means that the weights:

$$\boldsymbol{w_1} = \begin{bmatrix}\boldsymbol{w_{1,1}}\\ \boldsymbol{w_{2,1}}\end{bmatrix}$$

To reach the new point $\boldsymbol{x_1}$ in this new coordinate system are simply:

$$\boldsymbol{w_1} = \begin{bmatrix}\boldsymbol{w_{1,1}}\\ \boldsymbol{w_{2,1}}\end{bmatrix} = \begin{bmatrix}\mathbf{-2}\\ \mathbf{+1}\end{bmatrix}$$

This point $\boldsymbol{w_1}$ is called the **encoding** (or new representation) of $\boldsymbol{x_1}$ with respect to the basis $\{\boldsymbol{c_1}, \boldsymbol{c_2}\}$.

NOTE: We could also do it mathematically by calculating the **linear combination:**

$$\boldsymbol{w_{1,1}c_1} + \boldsymbol{w_{2,1}c_2} = \boldsymbol{x_1}$$

$$\boldsymbol{w_{1,1}}\begin{bmatrix}\mathbf{2}\\ \mathbf{1}\end{bmatrix} + \boldsymbol{w_{2,1}}\begin{bmatrix}\mathbf{1}\\ \mathbf{2}\end{bmatrix} = \begin{bmatrix}\mathbf{-3}\\ \mathbf{0}\end{bmatrix}$$

$$\begin{cases} 2w_{1,1} + 1w_{2,1} = -3 \\ 1w_{1,1} + 2w_{2,1} = 0 \end{cases} \qquad \begin{cases} 2w_{1,1} + 1w_{2,1} = -3 \\ w_{1,1} = -2w_{2,1} \end{cases} \qquad \begin{cases} -4w_{2,1} + 1w_{2,1} = -3 \\ w_{1,1} = -2w_{2,1} \end{cases}$$

$$\begin{cases} -3\, w_{2,1} = -3 \\ w_{1,1} = -2w_{2,1} \end{cases} \qquad \begin{cases} w_{2,1} = 1 \\ w_{1,1} = -2 \end{cases}$$

$$w_1 = \begin{bmatrix} w_{1,1} \\ w_{2,1} \end{bmatrix} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}$$

Subspaces and Dimensionality Reduction

Up to now, we stated that to span an N-dimensional space we need **at least N vectors** (and exactly N linearly independent vectors to form a basis). But:

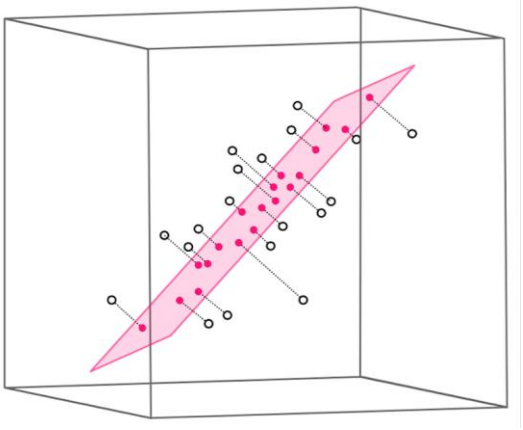
- **What happens if we choose fewer vectors?**
- **For instance, what if we take K vectors such that K < N?**

As a result, we can no longer perfectly represent every arbitrary point in the original space, leading to an inevitable loss of information for each data point x_p. This happens because K vectors can only span a K-dimensional subspace. Consequently, we must settle for an approximation, obtained by projecting the original points onto this lower-dimensional subspace.

This introduces a fundamental trade-off:

- **Dimensionality Reduction**: accepting a controlled loss of information in exchange for significantly reducing the complexity of the problem.

Example: Siamo in uno spazio 3D ($N = 3$), ma scegliamo soltanto $K = 2$ vettori, essendo cosi in grado di generare solo un sottospazio 2D, cioè un singolo foglio/piano cartesiano.

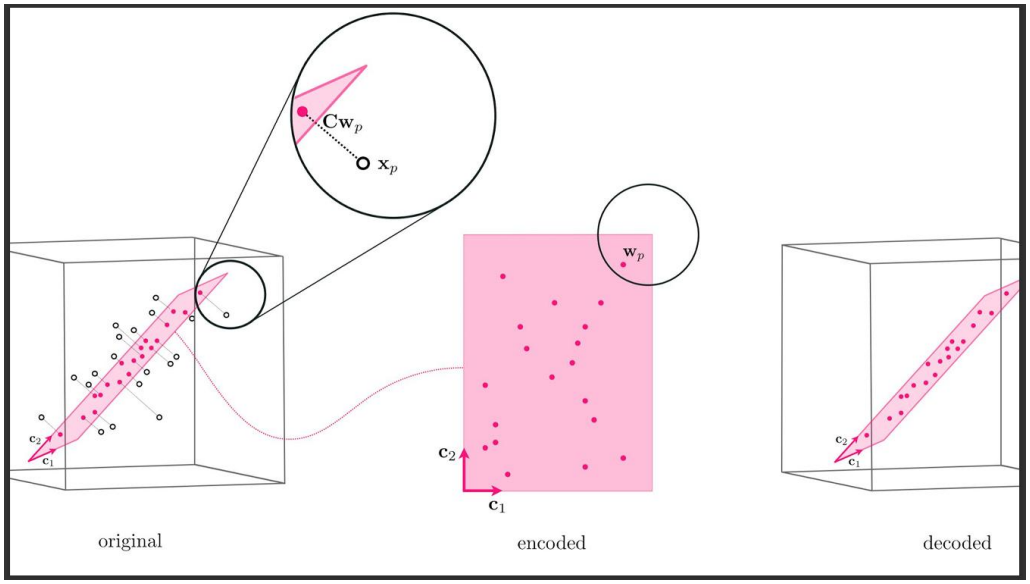


Ma, cosa succede ai dati?

- I punti bianchi rappresentano l'informazione originale nel piano 2D.
- Tali punti vengono poi proiettati perpendicolarmente sul piano 2D.
- I pallini rossi sono le proiezioni, cioè le migliori approssimazioni possibili dei punti su quella retta. I segmenti grigi tratteggiati rappresentano l'errore / informazione persa.

This process is called **projection**, because it is obtained by "dropping" a point perpendicularly onto the subspace spanned by the K basis elements.

- The resulting weight vector is the encoding of the point in this subspace.
- The decoding step reconstructs the point by combining the encoding with the K basis vectors, giving the projected point, the closest approximation of the original data point within the subspace.



Orthonormal Subspace

If our basis of K elements is orthonormal the property $\boldsymbol{C^T C = I}$ holds. Consequently, the encoding process is immediate. It does not require any system of equations:

$$\boldsymbol{C^T C w_p = C^T x_p}$$

We can just compute the dot product:

$$\boldsymbol{w_p = C^T x_p}$$

Principal Component Analysis (PCA)

Until now we only focused on how to project a point x_p from a space to another, trying to minimize the loss of information. However, we always assumed that the new basis *C* was given… **while, in the real world, nobody gives us the optimal basis to project our data onto…** We should find it by ourselves.

This is exactly what Principal Component Analysis (PCA) does: it is an algorithm that searches data to discover the absolute best spanning set (the principal components) for our dataset.

Minimize the Error

When we are projecting a point $\boldsymbol{x_p}$ from a space to another, we want to minimize the amount of lost information. Mathematical speaking:

The lost information is the distance from the new representation $\boldsymbol{w_p}$ and the original point $\boldsymbol{x_p}$.

NOTE: To calculate the distance between $\boldsymbol{w_p}$ and $\boldsymbol{x_p}$ we need them to be on the same space. So, we must multiply $\boldsymbol{w_p}$ by the basis $\boldsymbol{C}$, to get its "reconstructed version" on the original plane.

Let $\boldsymbol{h(w_p)}$ be the function that calculates that distance (the error). Our goal is to MINIMIZE this error $\boldsymbol{h(w_p)}$ to retain the maximum amount of original information.

> The function $\boldsymbol{h(w_p)}$ simply calculates the Euclidean Distance from $\boldsymbol{x_p}$ and the "reconstructed version" of $\boldsymbol{w_p}$ (denoted as $\boldsymbol{Cw_p}$, it is basically $\boldsymbol{w_p}$ brought back to the original space).
>
> We can formally define $\boldsymbol{h(w_p)}$ as:
>
> $$\boldsymbol{h(w_p) = ||Cw_p - x_p||_2^2}$$

Maximize the Variance

To fully understand WHY the Principal Component Analysis works, it is essential to understand the mathematical relationship between minimizing the reconstruction error and maximizing the variance.

If we mathematically expand the error function $\boldsymbol{h(w_p)}$ we get:

$$\boldsymbol{h(w_p) = ||Cw_p - x_p||_2^2 =}$$
$$\boldsymbol{= x_p^T C^T C C^T x_p - 2x_p^T C C^T x_p + x_p^T x_p}$$

And, assuming that the new basis C is orthogonal, we can use $\boldsymbol{C^T C = I}$ to get:

$$\boldsymbol{= -x_p^T C C^T x_p - x_p^T x_p =}$$
$$\boldsymbol{= -||C^T x_p||_2^2 + ||x_p||_2^2}$$

So, we find out that **the error depends mainly on 2 different elements**:

- **The Squared Projections onto the New Basis ($\boldsymbol{||C^T x_p||_2^2}$)**: It represents the portion of the original magnitude that is preserved after projection onto the new space. Since this term is subtracted in the expanded error equation, maximizing its value reduces the error.
- **The Magnitude of the Original Data Point ($\boldsymbol{||x_p||_2^2}$):** it represents the distance of the original data point $\boldsymbol{x_p}$ from the origin. Since this value only depends on the input, it remains constant during the process and cannot be altered by the new basis C.

So, since $\boldsymbol{||x_p||_2^2}$ only depends on the input and cannot be changed… the **only way to minimize the reconstruction error is to maximize the value of these projections ($\boldsymbol{||C^T x_p||_2^2}$)**.

From a statistical perspective, maximizing the squared projections across the entire dataset is mathematically equivalent to maximizing the variance of the data along the newly chosen directions. In simpler terms, finding the optimal subspace where we lose the least amount of information is identical to finding the axes along which our data is most spread out.

Covariance Matrix and Eigen-Decomposition

When we translate this conceptual goal into a formal optimization problem—specifically, maximizing the variance under the constraint that the new axes must be orthogonal to one another—the mathematics naturally lead us to a crucial statistical tool: the empirical covariance matrix ($\boldsymbol{\Sigma}$).

Def. This matrix compactly describes how the different features of our data vary together, effectively defining the overall shape and orientation of the **data cloud**.

The optimal solution to maximize the variance is achieved by performing an eigen-decomposition on this covariance matrix. This mathematical operation breaks the matrix down into its two fundamental components:

- **Eigenvectors**: These represent the principal directions (the new axes of our coordinate system) along which the data expands the most. In the context of this algorithm, these orthogonal vectors are exactly our Principal Components.
- **Eigenvalues**: These are scalar values associated with each eigenvector. They precisely quantify how much variance (and consequently, how much information) is captured along that specific direction.

By selecting the eigenvectors associated with the largest eigenvalues, PCA successfully constructs the optimal spanning set. This guarantees that the dimensions we choose to retain are the ones that describe the most significant and defining dynamics of the original dataset.

The PCA Protocol

Rather than focusing on the complex autoencoder optimization proofs, the practical application of PCA relies on analyzing the variance within the dataset. The protocol follows these core logical steps:

- **Compute the Covariance Matrix ($\boldsymbol{\Sigma}$):** The first step involves calculating the empirical covariance matrix for the dataset. This matrix compactly encodes how all features vary together. The diagonal entries represent the variance of individual features, while the off-diagonal entries represent the covariance (correlation) between different features. Geometrically, this matrix describes the shape, orientation, and elongation of the data cloud.
- **Determine Eigenvectors and Eigenvalues:** Next, we perform an eigenvalue decomposition on the covariance matrix. Because the covariance matrix is a real, symmetric matrix, its **eigenvectors** are perfectly orthogonal to one another. These eigenvectors point in the principal directions where the data spreads the most. The corresponding **eigenvalues** quantify exactly how much variance is captured along each of those specific directions.
- **Select Principal Components:** To reduce the dimensions of the data, the eigenvectors are sorted by their eigenvalues in descending order. By keeping only the top $\boldsymbol{K}$ components, we establish a new, reduced basis that captures the most significant variance in the dataset.

Exercise:

Consider the 2D dataset (not mean-centered):

$$x_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad x_3 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Do everything **by hand** (calculator allowed for arithmetic):

1. Compute the sample mean $\bar{x}$ and mean-center the data $c_p = x_p - \bar{x}$.
2. Compute the empirical covariance matrix $\Sigma = \frac{1}{P} \sum_{p=1}^{3} c_p c_p^\top$ (use $P = 3$).
3. Find the eigenvalues and eigenvectors of Σ.
4. Identify the first principal component v_1 (unit vector) and the second v_2.
5. Project the centered samples onto v_1 (compute the scalar scores $z_p = v_1^\top c_p$).
6. Reconstruct each sample from the 1D projection and compute the mean squared reconstruction error (MSE).
7. Compute the explained-variance ratio of the first PC (as a percentage).
8. Briefly interpret the sign of the off-diagonal covariance entry.

Solution: **TO DO**

Why is it useful?

PCA is widely considered the most fundamental **unsupervised learning** representation method, meaning it learns the inherent structure directly from the data distribution without requiring any external labels. Its utility in Data Science and Machine Learning includes:

- **Dimensionality Reduction:** By transforming data from a high-dimensional space into a lower-dimensional subspace, PCA effectively compresses the data while retaining as much critical information (variability) as possible.
- **Feature Engineering:** PCA creates entirely new, uncorrelated features from the original data. This removal of redundancy simplifies downstream machine learning tasks, improves overall model performance, and significantly reduces the risk of overfitting.
- **Pattern Recognition:** By learning a new basis that best explains data correlations, PCA helps practitioners uncover hidden patterns and detect underlying structures within complex datasets.

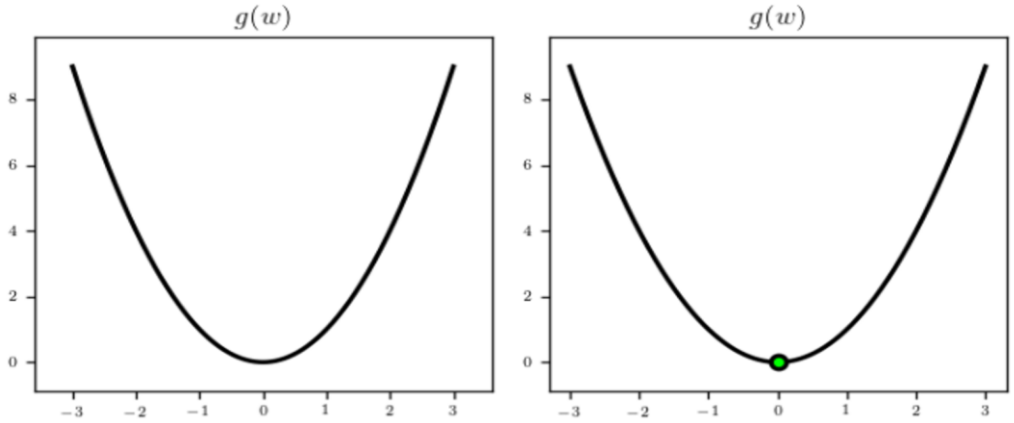
The Manifold Hypothesis

PCA is particularly successful because high-dimensional data rarely fills its entire space randomly; instead, it often lives near a lower-dimensional **manifold**. A manifold is a topological space that locally resembles standard Euclidean space (similar to how the curve of a circle can be mapped to a flat, one-dimensional line segment). By identifying this underlying linear manifold, PCA allows us to project complex data into a simplified space without losing its defining characteristics.

Optimization

Def. **Optimization refers to the task of either minimizing or maximizing an "objective function" $g(w)$ by altering its parameters w.**

In the context of Machine Learning and Data Science, we are typically interested in minimization, which is why the objective function is commonly referred to as a **loss function**.



The specific parameter value that achieves this minimum or maximum is denoted with *****, such as $\boldsymbol{w}^*$. Optimization landscapes contain two types of optimal points:

- **Global Optima:** The absolute smallest or largest value of the function across its entire domain.
- **Local Optima:** Points where the function's value is smaller (or larger) than all nearby points, but not necessarily the absolute best value overall.

These concepts of minima and maxima of a function are always related to each via multiplication by -1.

Finding the Minimum

To optimize a model (like finding the lowest error), we need to find the lowest point on a mathematical curve (the minimum). But how do we define "the lowest point," and more importantly, how do we tell a computer to find it?

Zero-Order Condition

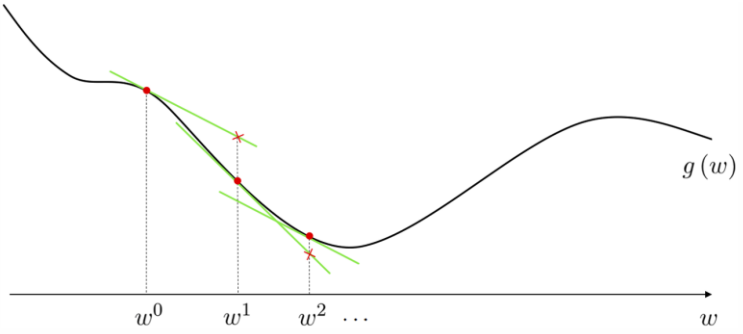
Zero-order simply means we are using zero derivatives. We are only looking at the original objective function itself $\boldsymbol{g(w)}$. We can write if formally as:

$$\boldsymbol{g(w^*) \leq g(w)} \textbf{ for all } \boldsymbol{w}$$

It gives us the absolute, strict definition of what a minimum is. However, it is not very practical. This is like the “brute force”. It means compute $\boldsymbol{g(w)}$ for all $\boldsymbol{w}$ to on the curve to see which one is the smallest. Absolutely impossible. It’s only a formal definition of “minimum”.

First-Order Conditions

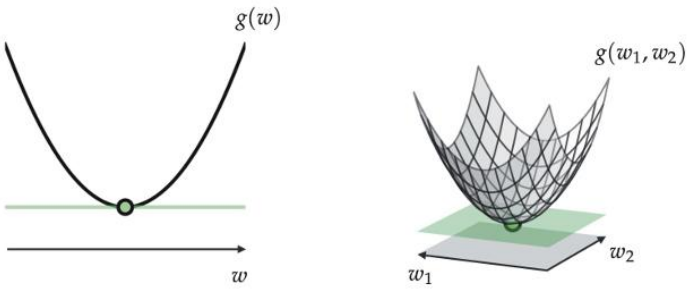
To actually find the minimum we use the "First-Order Condition". It means we are now looking at the first derivative of the function, denoted as $\frac{\boldsymbol{d}}{\boldsymbol{dw}}\boldsymbol{g(w)}$. **The first derivative represents the steepness(ripidità) of the curve at any given point**. Geometrically it means drawing a tangent line (a straight line touching the curve at exactly one point), to know the angle of that line.



Def. The first-order condition states that at **any minimum or maximum the derivate of the function is exactly zero**. Geometrically the tangent line is perfectly flat. Formally:

$$\frac{\boldsymbol{d}}{\boldsymbol{dw}}\boldsymbol{g(w)} = \boldsymbol{0}$$

So, instead of randomly checking every point on the curve (which the zero-order condition would require), we can use algebra or algorithms to find exactly where the derivative equals zero. Any point where the derivative is zero is a strong candidate for being our minimum.



IDEA: *"At the exact bottom of the valley (the minimum), the ground is perfectly flat, so the steepness is zero."*

Gradient

Def. In multiple dimensions the **derivate of the function** is named **gradient** and denoted as $\nabla \mathbf{g}(\mathbf{w})$. It is the **vector containing all the partial derivatives of the function at that specific point.**

So, the first order condition becomes:

$$\nabla \mathbf{g}(\mathbf{w}) = 0$$

The Challenge of Multiple Directions

In a one-dimensional space navigating an objective function is easy because there are only two possible directions to move: left or right. Moving in one direction will increase the function (ascent), while the other will decrease it (descent).

However, in multi-dimensional spaces there are infinitely many directions a step could take. The core problem in optimization is identifying which of these infinite directions will produce the largest possible increase or decrease in the function's value.

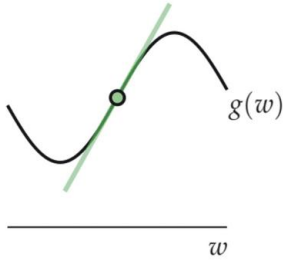
- **Direction of Steepest Ascent:** This is the specific unit direction that maximizes the increase of the function most rapidly. Mathematically, the direction of steepest ascent is given by the positive gradient vector, $+\nabla \boldsymbol{g}(\boldsymbol{w_0})$.
- **Direction of Steepest Descent:** Conversely, this is the direction where the function decreases most rapidly. It is the exact opposite of the steepest ascent, represented mathematically as $-\nabla \boldsymbol{g}(\boldsymbol{w_0})$.

Solving the Problem with the Gradient

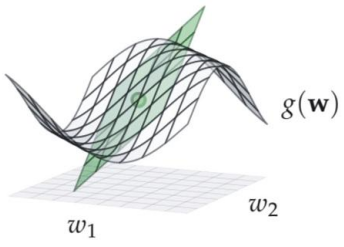
The problem of choosing a direction is solved by calculating the **gradient** $\nabla \boldsymbol{g}(\boldsymbol{w_0})$, which is the vector containing all the partial derivatives of the function at the specific point $\boldsymbol{w_0}$.

By its mathematical definition, the gradient vector naturally points in the exact direction of the maximum increase of the function. Because we know that near a point $\boldsymbol{w_0}$, the complex function $\boldsymbol{g}(\boldsymbol{w})$ behaves similarly to a flat tangent hyperplane, we can use this gradient information practically.

$$h(w) = g(w^0) + \frac{\mathrm{d}}{\mathrm{d}w} g(w^0)\left(w - w^0\right)$$



$$h(\mathbf{w}) = g(\mathbf{w}^0) + \nabla g(\mathbf{w}^0)^T\left(\mathbf{w} - \mathbf{w}^0\right)$$



If our goal is to minimize a loss function in Machine Learning, we do not have to guess which way to go; we simply calculate the gradient and travel strictly along the negative gradient path, $-\nabla \boldsymbol{g}(\boldsymbol{w})$. This straightforward geometric property is the foundation of the **Gradient Descent Algorithm**, allowing computers to systematically step down the steepest slope until they reach a local minimum.

Automatic Differentiation

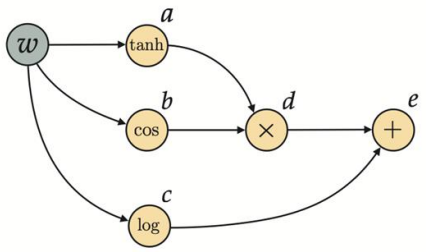
In machine learning any complex algebraic function can be systematically broken down into a combination of elementary functions (e.g., $\boldsymbol{sin(x)}$, $\boldsymbol{e^{(x)}}$, $\boldsymbol{log(x)}$, etc…) and fundamental operations (e.g., +, *).

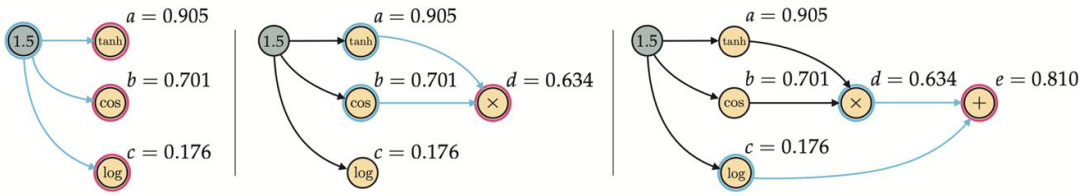
Computation Graphs

Def. **A computation graph is a structural tool used to organize the decomposition,** enabling a computer to programmatically evaluate the function.

When reading a computation graph, the process starts at the input node on the left and flows to the right until the full function is computed. The topology of this graph relies on **parent nodes** (where an edge originates) passing information to **child nodes** (where the edge points). Ultimately, every child node is evaluated as a function of its parents, meaning the entire graph evaluates the original input as it flows forward.

$$g(w) = \tanh(w)\cos(w) + \log(w).$$





Forward Mode Automatic Differentiation

Def. **Forward Mode Automatic Differentiation (AD)** applies this graph structure to compute gradients.

The algorithm sweeps forward through the graph, evaluating the gradient of each node with respect to the original input. However, although it is highly reliable and easily programmable, it requires each node to compute a complete gradient with respect to all inputs. In fully connected neural networks with millions of inputs, this results in a massive amount of computational waste.

Backpropagation - Reverse Mode Automatic Differentiation

To resolve the Forward Mode computational waste, the deep learning industry relies on Reverse Mode AD, also referred to as **Backpropagation**. This approach splits the computation into two distinct phases:

- **Forward Pass:** The algorithm traverses the graph from the input nodes to the output node (left to right), computing only the necessary partial derivatives rather than the full gradient.
- **Backward Pass:** The algorithm then traverses the graph in reverse, starting from the output node and moving recursively back to the inputs. At each step, it applies the chain rule by multiplying the parent's partial derivative by the child's partial derivative.

Backpropagation requires the system to store the computation graph in memory, resulting in a slight memory overhead. However, it completely avoids the wasted computation of irrelevant derivatives, making it the universal standard for automatic gradient computation.

Gradient Descent

Now that we know the backpropagation we can finally use it to efficiently compute the gradient even in fully connected neural networks. Once Backpropagation provides this gradient vector, **Gradient Descent** takes over as the optimization algorithm that actually uses this information to update the model's parameters and **minimize the loss function**.

Def. The **Gradient Descent** is an iterative optimization algorithm that leverages the negative gradient to step toward a local minimum.

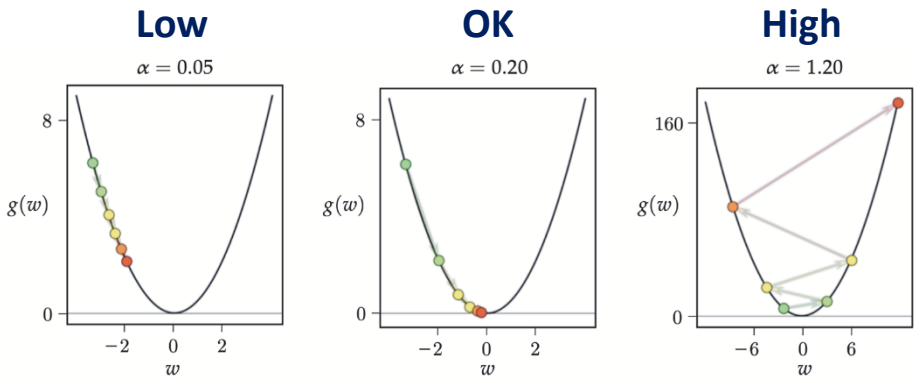
The mathematical update rule for Gradient Descent is:

$$\boldsymbol{w^k = w^{k-1} - \alpha \nabla g\left(w^{k-1}\right)}$$

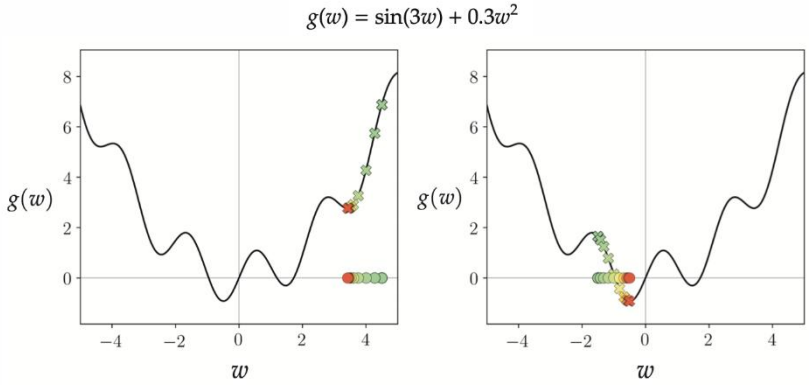
Where:

- $\boldsymbol{w^k}$ is the new, updated parameter.
- $\boldsymbol{\nabla g\left(w^{k-1}\right)}$ is the computed gradient at the current position.
- $\boldsymbol{\alpha}$ represents the **learning rate** (or step size).

In this way Gradient Descent guarantees a descent direction, ensuring that the model makes continuous progress toward a minimum… However, it works well as long as the step size is appropriate…



…And, when dealing with complex, nonconvex functions Gradient Descent can easily become trapped in sub-optimal local minima or saddle points depending entirely on where the initial starting point ($\boldsymbol{w_0}$) is placed.



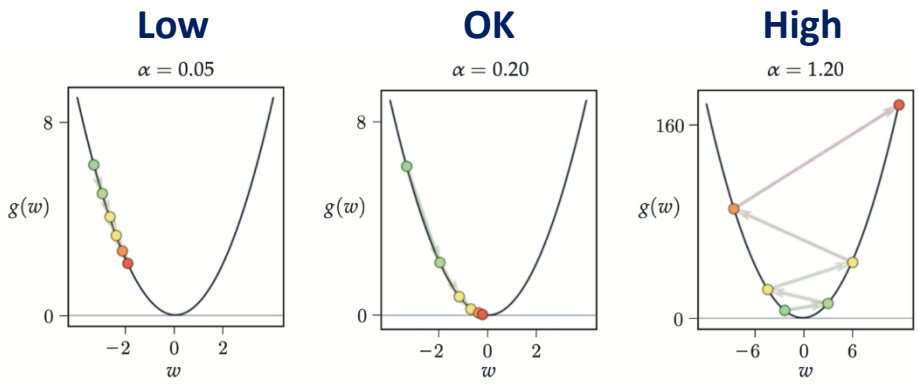
NOTE: For strictly convex functions, this is not actually an issue, as all local minima are guaranteed to be global minima.

Learning Rate

Def. **The learning rate (α) dictates the magnitude of the steps the Gradient Descent Algorithm**

It must be chosen very carefully.

- **If α is too small**, the algorithm makes incredibly slow progress.
- **If α is too large**, the algorithm might jump over the minimum, causing it to oscillate wildly or diverge upwards. However, it can sometimes be beneficial, as the "hopping around" behaviour might help the algorithm jump out of a poor local minimum to reach a better global minimum.



There are two primary strategies to define α:

- **Fixed Step Length:** The learning rate remains constant during the process. While simple, a fixed rate can sometimes cause the algorithm to get stuck and continuously oscillate around the exact minimum without ever settling.
- **Diminishing Step Length:** The learning rate progressively decreases as the iterations advance. This allows the algorithm to take large exploratory steps initially and then fine-tune its position as it approaches the minimum, ensuring a stable convergence.

The algorithm stops when it reaches a stationary point. In practice, stopping criteria include:

1. The magnitude of the gradient approaches zero.
2. The steps taken ($\boldsymbol{w^k - w^{k-1}}$) become negligibly small.
3. The change in the overall loss function value is negligible.
4. A hard limit on the maximum number of iterations is reached.

Cost Function History Plots

To monitor the optimization process, data scientists use **Cost Function History Plots** to track how the loss value changes over successive iterations. These plots show exactly how the cost function evolves during Gradient Descent, making them an essential debugging tool. They allow data scientists to visually detect if a model is diverging, stagnating, or successfully converging, which directly informs how to tune the learning rate and other hyperparameters Some oscillation in these plots is actually acceptable, as it can occasionally help the algorithm escape a shallow minimum.

Weaknesses of Gradient Descent

Because a gradient vector contains both a direction and a magnitude, Gradient Descent is prone to two primary mechanical flaws:

- **Zig-Zagging:** In landscapes that feature narrow valleys, the contour lines are highly elongated. Because the gradient always points perpendicular to these contours, successive steps constantly alternate directions, resulting in a zig-zag motion that severely slows down convergence.
- **Slow-Crawling:** As the algorithm approaches a minimum or a saddle point, the gradient magnitude naturally shrinks near zero. This forces the algorithm to take vanishingly small steps, which can cause the optimization process to halt prematurely at a saddle point before finding the true minimum.

Machine Learning

Given a dataset $\boldsymbol{Z}$, a machine learning algorithm aims to output a specific model, denoted as $\boldsymbol{f^*}$, that best approximates the provided data.

Data → ML Algorithm → Model

$\boldsymbol{Z}$ → ML Algorithm → $\boldsymbol{f^*}$

This idea is based the assumption that there is a real (but unknown) mathematical function *f*, that originally generated the dataset. However, we cannot see this function directly so we try to find another function *f** that approximates *f*, inside a predefined **family of candidate models**, which we call $\boldsymbol{F}$.

However, there is a problem. The true function *f* is usually very complex, so it often does not belong to the family of models we have chosen. This means we can only get close to it, but never reach it exactly. To solve this, engineers and data scientists have to make two key design decisions:

1. How do we mathematically define what "approximating" means?
2. What specific family of candidate models $\boldsymbol{F}$ should we choose for the problem at hand?

Supervised Learning

In a supervised learning scenario, we have access to a dataset $\boldsymbol{\mathcal{D}}$ composed of input-output pairs.

This is formally defined as $\boldsymbol{\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}}$. Each individual data point is sampled independently and identically distributed (i.i.d.) from an unknown joint probability distribution $\boldsymbol{\{(x_i, y_i)\}_{i=1}^{n} \sim p(x, y)}$.

Then, depending on the type of label, a supervised problem can be either:

- **Regression**: y ∈ ℝ (e.g. predicting the price of a house)
- **Classification**: y in a finite set of classes (e.g. predict if the image shows a cat, a dog or a bird).

The ultimate task in supervised learning is to find a function that not only accurately models the provided training data but also **generalizes** well. Generalization means the model can make reliable predictions for new input locations that were never seen during the training phase.

Loss Function

To evaluate how well our model is performing, we introduce a **loss function** $\boldsymbol{l(f(x), y)}$, that represents a mathematical penalty for using $\boldsymbol{f(x)}$ instead of the true value $\boldsymbol{y}$.

Risk Minimization vs. Empirical Risk Minimization

Ideally, we want to find the best possible function $\boldsymbol{f^*}$ that minimizes the expected loss across the entire distribution $\boldsymbol{p(x, y)}$. This theoretical concept is known as **Risk Minimization**, written mathematically as:

$$\boldsymbol{f^* = \arg\min_f \mathbb{E}_{p(x,y)} [l(f(x), y)]}$$

However, we do not know the true distribution $\boldsymbol{p(x, y)}$… which means we cannot compute this exact expectation directly. To solve this, we must approximate the theoretical expectation using a simple average over the training data. This practical approach is called **Empirical Risk Minimization**, defined as:

$$\boldsymbol{f^* = \arg\min_f \frac{1}{n} \sum_{i=1}^{n} l(f(x_i), y_i).}$$

Linear Models

One of the most foundational model families to choose from is the space of linear functions.

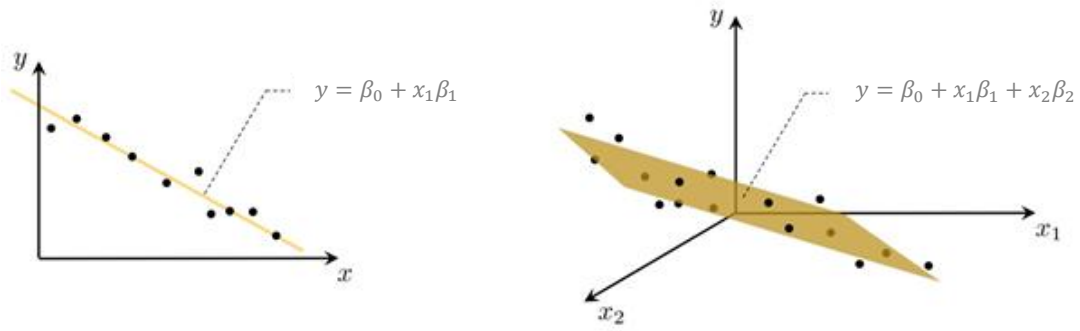
Def. For an input vector $\boldsymbol{x} \in \mathbb{R}^{\boldsymbol{d}}$, a linear model $\boldsymbol{f_\beta(x)}$ is calculated using the dot product between the inputs and the model's parameters (weights):

$$\boldsymbol{f_\beta(x) = \beta^\top x = \beta_0 x_0 + \cdots + \beta_d x_d \; = \; y}$$

In this equation:

- $\boldsymbol{\beta} \in \mathbb{R}^{\boldsymbol{d}}$ is the **parameter vector** (often called weights) that the algorithm must learn.
- $\boldsymbol{y = f_\beta(x)}$ is the predicted **label** or output, which in this context is a simple scalar value.

Geometrically, linear models create flat decision boundaries. If the data has one input feature, the linear model $\boldsymbol{(y = \beta_0 + x_1\beta_1)}$ takes the shape of a straight line in a 2D space. If the data has two input features, the model $\boldsymbol{(y = \beta_0 + x_1\beta_1 + x_2\beta_2)}$ forms a two-dimensional flat plane in a 3D space. And so on…



Bias (or Intercept)

Def. The bias, typically denoted as $\boldsymbol{\beta_0}$, is a fundamental parameter that shifts the model's decision boundary (such as a line or a plane) away from the origin.

Without a bias term $\boldsymbol{\beta_0}$, a linear model would be strictly forced to pass exactly through the origin point $(\boldsymbol{0}, \boldsymbol{0})$, making the mode unable to fit most real-world data.

Conceptually, a standard **linear prediction** is calculated as:

Linear Prediction = (Weights × Inputs) + Bias

However, when models scale to include many input features $(\boldsymbol{x_1}, \boldsymbol{x_2}, \ldots, \boldsymbol{x_d})$, computers prefer to calculate these predictions using a single, fast vector multiplication. So, computer scientists decided to eliminate the separate addition step incorporating the bias directly into the main parameter vector.

But… every weight needs an input to multiply with! So, they invented a fake input feature $\boldsymbol{x_0}$ always set at 1 (such that $\boldsymbol{\beta_0 \times 1 = \beta_0}$).

Regression Metrics

To train our linear model, we need a specific loss function to measure if $y_i \approx \beta^\top x_i$. A standard and highly effective choice is the **Mean Squared Error (MSE)**. The MSE calculates the average of the squared differences between the true labels and the model's predictions:

$$L(\beta; Z) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \beta^\top x_i)^2.$$

The MSE is widely used because it is computationally convenient and yields excellent results in practice. By squaring the individual errors, it guarantees that positive and negative deviations do not cancel each other out; all errors contribute positively to the final metric. Additionally, squaring the values acts as a strict penalty system, giving significantly greater weight (and therefore harsher penalties) to larger prediction errors compared to smaller ones.

NOTE: To perform computations efficiently, especially on modern hardware, machine learning algorithms utilize linear algebra to "vectorize" operations.

Instead of iterating through individual data points, we can group all target values into a single vector Y and compute all predictions simultaneously as a matrix multiplication $X\beta$. Consequently, the empirical risk (the loss function) can be elegantly rewritten using the L_2 norm of the residual vector:

$$L(\beta; Z) = \frac{1}{n}||Y - X\beta||_2^2$$

While the Mean Squared Error (MSE) is a standard loss function for training linear models, data scientists frequently rely on alternative metrics to evaluate a model's performance from different perspectives. These metrics provide different insights into how well the predictions align with the true data:

- **Mean Absolute Error (MAE):** This metric measures the average absolute difference between the predicted values ($\hat{y}_i$) and the true target values (y_i). Unlike MSE, which heavily penalizes large errors by squaring them, MAE treats all errors proportionally. It is defined as:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|$$

- **Mean Relative Error (MRE):** Sometimes, knowing the absolute error is not enough; we need to understand the error relative to the scale of the target variable. The MRE expresses errors as a ratio of the true magnitude. It is calculated as:

$$MRE = \frac{1}{n}\sum_{i=1}^{n}\frac{|\hat{y}_i - y_i|}{|y_i|}$$

- **Coefficient of Determination (R^2):** This statistical measure indicates the proportion of the variance in the dependent variable that is predictable from the independent variables. An R^2 value of 1 represents a perfectly accurate prediction. Generally, higher values indicate a better model fit. It is computed as:

$$R^2 = 1 - \frac{MSE}{Variance(y)}$$

Optimizing Linear Models

While defining a model family and a loss function provides the mathematical framework, we need a systematic computational method to actually find the optimal parameters (or weights) that minimize the model's prediction error. In the following paragraphers, we'll explore the main optimization algorithms used to train linear models.

Linear Regression Algorithm

Def. The linear regression algorithm provides a formalized mathematical process to optimize the parameter vector (weights).

The algorithm (as every supervised learning algorithm) begins with an input dataset $\mathcal{D} = \{(\boldsymbol{x_i}, \boldsymbol{y_i})\}_{i=1}^{N}$, which contains the input features and their corresponding target labels.

Then, it searches for the optimal parameter vector $\boldsymbol{\beta_Z^*}$ that **minimizes the Mean Squared Error (MSE)** loss function over the entire dataset. Mathematically, this is expressed as

$$\boldsymbol{\beta_Z^*} = \arg\min_{\boldsymbol{\beta} \in \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^{n} (\boldsymbol{y_i} - \boldsymbol{\beta}^\top \boldsymbol{x_i})^2.$$

Once these optimal parameters are found, the algorithm outputs the final predictive model, formulated as:

$$\boldsymbol{f_{\beta^*}(x)} = \boldsymbol{\beta^{*\top} x}$$

Gradient Descent

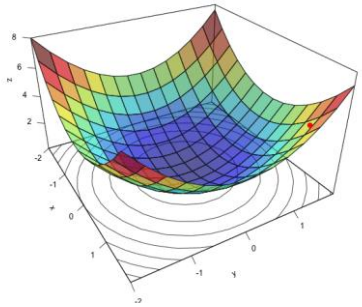
Gradient Descent is an iterative optimization algorithm used to find the optimal parameters (weights) that minimize a convex loss function. In simple terms, it is the method our machine learning model uses to learn the best possible weights by systematically minimizing its prediction error. To compute it, we rely on the concept of the **gradient**, denoted as $\boldsymbol{\nabla g(w)}$**.**

Def. The gradient $\boldsymbol{\nabla g(w)}$ is a mathematical vector that contains all the partial derivatives of a function with respect to its parameters.

$$\boldsymbol{\nabla g(w)[i] \; = \; i - th \; derivate \; of \; the \; function \; g(w)}$$

It describes both the direction and the rate of change of a given function $\boldsymbol{g(w)}$.

Example: Imagine you are blindfolded at the top of a hill, and your goal is to reach the absolute lowest point in the valley (which represents the minimum error). Because you cannot see, you use your feet to feel the slope of the ground (the gradient) to decide which way to step.



Because the gradient points in the direction of the steepest ascent(il punto in cui si sale piu velocemente), algorithms can use the opposite direction of the gradient to move toward the minimum error.

Protocol:

1. At each step, the algorithm calculates the **partial derivatives** (the gradient, denoted as ∇) of the loss function. The derivative tells the algorithm the direction and the rate at which the error is increasing.
2. Since the goal is to minimize the error, the algorithm must move in the opposite direction of the gradient. This is why the algorithm subtracts the gradient from the current weights.

$$\boldsymbol{w^k = w^{k-1} - \alpha \nabla g(w^{k-1})}$$

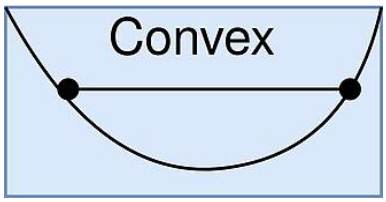
In this equation, $\boldsymbol{\alpha}$ represents the learning rate (the step size). If the gradient tells the algorithm the direction to move, the learning rate dictates how large of a step to take

3. Iterate this process until it reaches the bottom of the valley, where the slope is practically flat, reaching a state known as **convergence**.

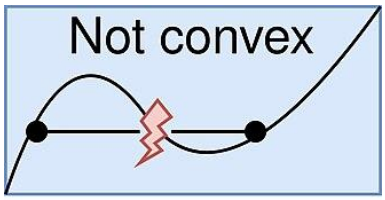
Convexity

Gradient descent can be applied to any differentiable function, but the reliability of the result depends heavily on the function's shape:

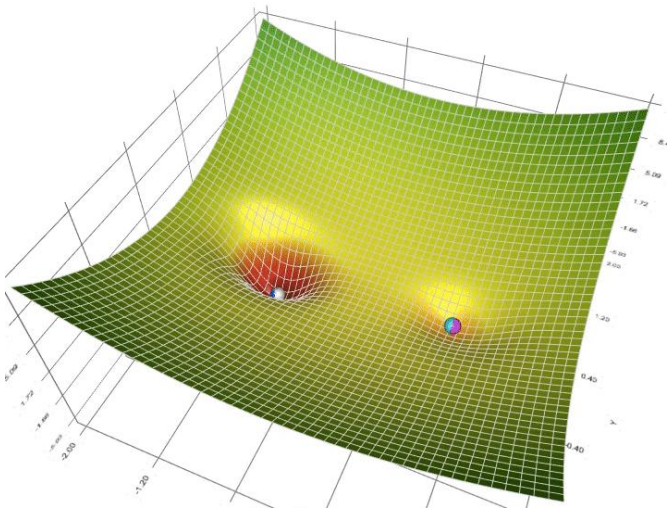
- **A convex function is uniquely bowl-shaped**. Mathematically, if you draw a straight-line segment between any two points on its curve, that line will always lie on or above the curve itself. For convex functions (like the Mean Squared Error used in linear regression), Gradient Descent is guaranteed to find the global minimum (the absolute lowest possible error).



- **For non-convex functions** (which are extremely common in more complex models like deep neural networks), the mathematical landscape features multiple distinct valleys. In these scenarios, Gradient Descent might converge on a local minimum (a low point that is optimal within its immediate vicinity, but is not the absolute lowest point overall).



Example:



Calculate the Partial Derivatives

However, to apply gradient descent to our linear model, **we must compute the partial derivatives of the loss function** (in our case MSE).

For a simple linear regression with an intercept ($\hat{\beta}_0$) and one feature weight ($\hat{\beta}_1$), the loss function is:

$$\boldsymbol{MSE(\hat{\beta}_0, \hat{\beta}_1) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i)\right)^2}.$$

Calculating the derivatives of this function indicates the specific direction in which the prediction error is increasing. The partial derivatives with respect to each parameter are computed as follows:

$$\boldsymbol{\frac{\partial MSE}{\partial \hat{\beta}_1} = -\frac{2}{n}\sum_{i=1}^{n} x_i\,(y_i - \hat{y}_i)}$$

Because the derivatives point toward the maximum error, gradient descent uses the *negative* values of these derivatives to iteratively update and adjust the model parameters. The algorithm repeats the following update rules until it converges on the minimum error:

Variants of Gradient Descent

Depending on how the data is processed during each iterative update, there are three primary variations of the gradient descent algorithm:

- **Batch Gradient Descent:** This approach uses the entire dataset to compute the gradient at every single step. While this guarantees a very stable convergence toward the minimum, it is extremely slow and memory-intensive when dealing with large datasets.
- **Stochastic Gradient Descent (SGD):** In contrast to the batch method, SGD uses only a single, randomly chosen data point to compute the gradient at each iteration. This introduces noisy updates that can cause the error to oscillate, but this erratic behavior has the advantage of helping the algorithm escape sub-optimal local minima.
- **Mini-batch Gradient Descent:** This is a hybrid approach that computes the gradient using a small, fixed-size subset (a mini-batch) of the data per update. It is highly efficient, offers stable convergence, and is the most widely used variation in practical machine learning applications.

Normal Equation

If the loss function is perfectly convex and the model is strictly linear, there is an alternative to the iterative approach. We can use a direct, closed-form mathematical solution to find the minimum in a single step, known as the Normal Equation:

$$\boldsymbol{\beta = (X^\top X)^{-1} X^\top y}$$

In this formula, $\boldsymbol{X}$ represents the matrix of all input features, and $\boldsymbol{y}$ represents the vector of target values.

However, while finding the exact optimal parameters in a single mathematical step sounds ideal, computing the inverse of the matrix $\boldsymbol{(X^\top X)^{-1}}$ becomes extremely slow and computationally expensive when a dataset has thousands of features.

In addition, the Normal Equation strongly relies on the assumption that the matrix $X^\top X$ is invertible, that is not always true and can lead the system to have multiple non-unique solutions.

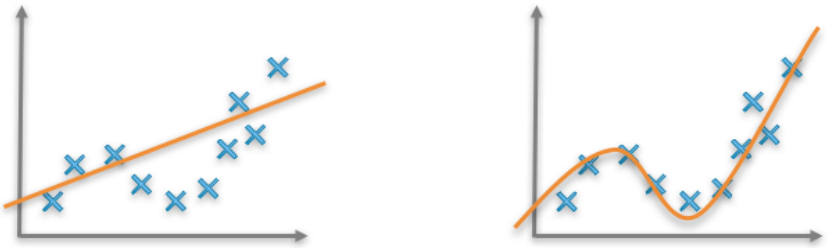
The combination of these 2 factors explains is why Gradient Descent remains the standard choice for large-scale machine learning problems.

Non-Linear Models

Strictly linear models are inherently limited because they can only draw straight lines or flat planes, which **do** not capture most of the real-world relationships. To overcome this limitation, we utilize **non-linear models** that expand the standard linear model family by applying Feature Maps, that transform the original input space into higher-dimensional representations, enabling the model to learn curves and complex patterns while retaining the underlying mathematical structure of linear optimization.

Feature Maps

Def. A feature map, denoted as $\boldsymbol{\phi(x)}$, allows us to **encode prior knowledge**(Conoscenza pregressa) by transforming the original input space into a new, often higher-dimensional feature space.



Linear function | Non-linear function

For example, if we believe the relationship in the data is quadratic rather than strictly linear, we can define a feature map that squares the inputs:

$$\boldsymbol{\phi(x)} = \begin{bmatrix} \boldsymbol{x} \\ \boldsymbol{x^2} \end{bmatrix}$$

By applying standard linear regression on these transformed features, the model's equation becomes a polynomial:

$$\boldsymbol{f_\beta(x) = \beta_1 x + \beta_2 x^2}$$

In this context, the terms "input" (the raw data $\boldsymbol{x}$) and "features" (the mapped data $\boldsymbol{\phi(x)}$) are distinct concepts, although they are often used interchangeably in practice. This technique demonstrates that "linear" regression only requires the model to be linear with respect to its parameters ($\boldsymbol{\beta}$), not its raw inputs.

Polynomial Features

One of the most common approaches is the use of **Polynomial Features** to represent correlations in the original inputs.

For example, let's imagine that we want to predict the price of a house through 2 different inputs:

- $\boldsymbol{x_1}$ = Size of the house (in square meters)
- $\boldsymbol{x_2}$ = Age of the house (in years)

Combining these 2 raw inputs in a single vector $\boldsymbol{x}$ and giving it to a **quadratic feature map $\boldsymbol{\phi(x)}$,** the math trick generates 6 new features:

$$\boldsymbol{\phi(x) = \left[1, x_1, x_2, x_1^2, x_1 x_2, x_2^2\right]}$$

the interceptor (1), the original inputs ($\boldsymbol{x_1, x_2}$), their squares ($\boldsymbol{x_1^2, x_2^2}$), and their combination ($\boldsymbol{x_1 x_2}$). We turned 2 raw inputs into 6 mathematical features that better describes the relations among the data.

NOTE: We choose a "quadratic" transformation (raising inputs to the power of 2) simply because we suspect the data does not form a straight line. If you look at the real world, relationships are rarely perfectly straight. For example, a house's price drops quickly as it ages from 1 to 20 years, but the price drop slows down between 80 and 100 years. A straight line cannot capture this slowing down; it needs a curve.

NOTE: We don't have to use always a quadratic! If the data was even more complex and wavy, we could use a "cubic" mapping (power of 3), which would generate even more features like $\mathbf{x}_1^3$. The quadratic map is just a very common, simple choice to start with.

We could also use higher powers, but this directly leads us to the concept of Model Capacity and Overfitting. If we choose a polynomial of degree 10, the model gains the flexibility to draw wildly fluctuating, squiggly lines. This extremely high capacity almost guarantees severe overfitting; the model will bend sharply just to touch every single training point, completely failing to learn the true underlying trend. Because of this risk, data scientists usually stop at degree 2 (quadratic) or degree 3 (cubic) unless the specific problem explicitly demands higher complexity.

Overfitting vs. Underfitting

Seeing the power of polynomial feature maps, a logical question arises:

> *"Why not always use an enormous number of features to capture every tiny detail in the data?"*

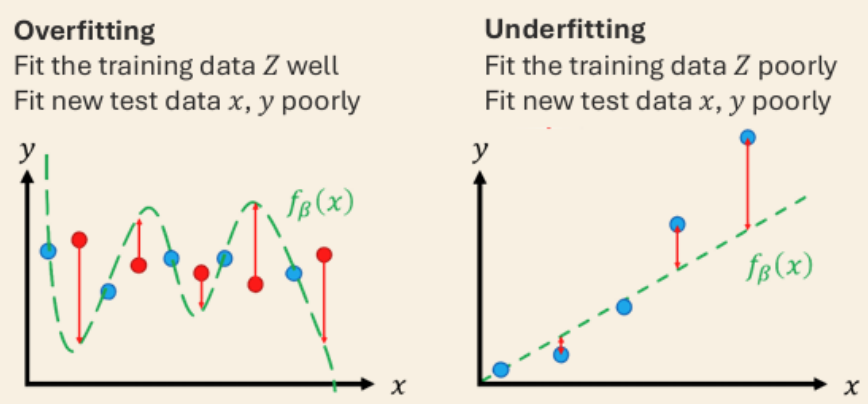
To answer this, we must distinguish between two phases of data usage in machine learning:

- **Training data ($Z = \{x, y\}$):** The subset of examples explicitly used by the algorithm to adjust its weights and fit the model.
- **Test data:** A separate set of new, unseen inputs whose labels we want to predict.

If we give our model too many complex features, it will draw a highly erratic curve that perfectly passes through every single training point but completely misses the actual underlying trend, leading to massive errors when evaluated on the unseen test data.

This behavior introduces two of the most critical failure modes in machine learning:

- **Overfitting:** This occurs when a model fits the training data exceptionally well but fits new test data poorly. The model has essentially memorized the statistical noise and random fluctuations of the training set rather than learning the general pattern.
- **Underfitting:** This happens when a model is too simplistic. It fails to capture the underlying pattern of the data, resulting in poor performance on both the training data and the new test data.



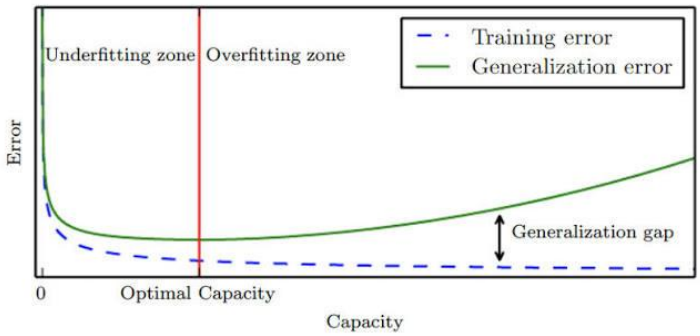
Model Capacity

To formalize this concept, data scientists refer to a **Capacity**, which is a measure of the "complexity" of the data the model is capable of fitting.

Choosing the right capacity involves a delicate balancing act:

- **High-Capacity Models:** These models have the flexibility to fit highly complex data distributions. However, they are highly prone to overfitting, a condition associated with high **variance** (meaning the model's predictions change drastically depending on the specific training data used).
- **Low-Capacity Models:** These models can only fit very simple relationships (like a strict straight line). They are highly likely to underfit the data, a condition associated with high **bias** (meaning the model stubbornly makes systematic errors because it is too rigid).

When observing the optimization process over time (or as model capacity increases), the training loss will predictably continue to decrease toward zero as the model learns. However, the test loss will initially drop and then eventually curve back upwards, forming a U-shape. The **ideal** model capacity is found exactly at the lowest point of the test loss curve, representing the perfect middle ground between underfitting and overfitting.



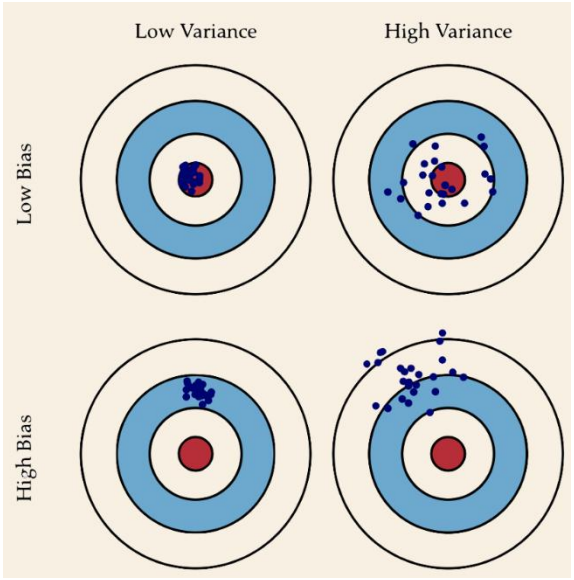
Bias and Variance

To deeply understand why models fail to generalize, we analyse errors through the lens of **Bias** and **Variance**.

Imagine a theoretical scenario where we draw $\boldsymbol{k}$ different, independent training datasets from the exact same underlying real-world distribution. We then train $\boldsymbol{k}$ different models ($\boldsymbol{f}_{\boldsymbol{\beta}_1}, \ldots, \boldsymbol{f}_{\boldsymbol{\beta}_k}$), one on each dataset.

Using the analogy of a dartboard (where the bullseye is the true underlying function we want to learn):

- **Variance:** This measures how much the k fitted models differ from one another depending on which specific dataset they were trained on. High Variance means the model is overly sensitive to the random noise in the training data, which is the primary cause of overfitting.
- **Bias:** This measures how much the average prediction of all k models deviate from the true target function. High Bias (darts clustered together, but far from the bullseye) means the model is fundamentally too simple to capture the true relationship, which results in errors and underfitting.



But, as we already said, we could use feature maps to transform our data into a higher-dimensional space which allows a simple linear model to learn complex, non-linear relationships. However, increasing the model's capacity inherently raises also the risk of overfitting.

To defend against overfitting, the size of the dataset is critical. As a general rule, the number of training samples (n) should be substantially larger than the new feature dimension (d').

Mathematically, a model requires approximately $n \approx d' \log(d')$ samples to generalize well. Ultimately, project success rarely hinges only on selecting the perfect model; the real work lies in thoughtful data cleaning and feature engineering to ensure the model has high-quality information to learn from.

Mathematical Decomposition of Expected Error

To truly understand generalization, we return to the concept of **Risk Minimization**, where the ideal model f^* minimizes the expected error (also known as generalization error or prediction error).

The Expected Error quantifies exactly how accurately a model predicts outcomes for new, unseen data. For linear regression, this is calculated as the mean squared difference between the true target y and the model's prediction $\hat{f}(x)$, averaged over all possible training sets.

$$E\left[\left(y - \hat{f}(x)\right)^2\right] = \overbrace{\left(E[\hat{f}(x)] - f(x)\right)^2}^{\text{Bias}} + \overbrace{E\left[\left(\hat{f}(x) - E[\hat{f}(x)]\right)^2\right]}^{\text{Variance}} + \overbrace{\sigma^2}^{\text{Irreducible Error}}$$

This expected error can be mathematically decomposed into three fundamental components:

1. **Bias:** This measures how far the average model prediction is from the true, underlying function $f(x)$.
2. **Variance:** This measures how much the specific prediction $\hat{f}(x)$ would change if the model were trained on completely different data samples.
3. **Irreducible Error:** This represents the natural noise inherent in the data generation process itself, which no model can ever eliminate.

Deriving the Decomposition (The Proof)

To prove this decomposition, we start with our basic assumptions: the true target value is generated by a true function plus some random noise ($\boldsymbol{y = f(x) + \epsilon}$). We assume this noise has a mean of zero ($\boldsymbol{E[\epsilon] = 0}$) and a constant variance ($\boldsymbol{Var(\epsilon) = \sigma^2}$).

We begin with the Mean Squared Error equation and substitute $\boldsymbol{y}$ with its assumed true form:

$$\boldsymbol{MSE = E\left[\left(f(x) + \epsilon - \hat{f}(x)\right)^2\right]}$$

Next, we employ a mathematical trick: we simultaneously add and subtract the expected prediction $\boldsymbol{E[\hat{f}(x)]}$ inside the squared term:

$$\boldsymbol{MSE = E\left[\left(\left(f(x) - E[\hat{f}(x)]\right) + \left(E[\hat{f}(x)] - \hat{f}(x)\right) + \epsilon\right)^2\right]}$$

We can now treat this as a trinomial $\boldsymbol{(A + B + C)^2}$, which expands into $\boldsymbol{A^2 + B^2 + C^2 + 2AB + 2AC + 2BC}$. The quadratic terms (the squared components) directly form our three main error sources:

- $\boldsymbol{A^2 = E\left[\left(f(x) - E[\hat{f}(x)]\right)^2\right]} = \text{Bias}^2$
- $\boldsymbol{B^2 = E\left[\left(E[\hat{f}(x)] - \hat{f}(x)\right)^2\right]} = \text{Variance}$
- $\boldsymbol{C^2 = E[\epsilon^2] = \sigma^2}$ (Irreducible Error)

The remaining cross-product terms elegantly vanish to zero for the following statistical reasons:

- **The Bias-Variance Cross-Product ($\boldsymbol{2AB}$):** This term is $\boldsymbol{2\mathbb{E}\left[\left(f(x) - \mathbb{E}[\hat{f}(x)]\right) \cdot \left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)]\right)\right]}$. The first factor (the bias) acts as a constant for a fixed input $\boldsymbol{x}$ and can be pulled outside the expectation. The second factor measures the model's random fluctuations around its own mean. By definition, the expected value of these random fluctuations across all training sets is exactly zero ($\boldsymbol{\mathbb{E}\left[\hat{f}(x) - \mathbb{E}[\hat{f}(x)]\right] = 0}$), causing the entire term to evaluate to zero.
- **The Model-Noise Cross-Products ($\boldsymbol{2AC}$ and $\boldsymbol{2BC}$):** These terms involve the interaction between the model's error and the natural noise $\boldsymbol{\epsilon}$. Because the model's predictions $\boldsymbol{\hat{f}(x)}$ depend entirely on the training data, they are completely independent of the random noise generated for new predictions. For independent variables, the expectation of their product is the product of their expectations ($\mathbb{E}[\text{model term} \cdot \boldsymbol{\epsilon}] = \mathbb{E}[\text{model term}] \cdot \mathbb{E}[\boldsymbol{\epsilon}]$). Since the noise has a zero mean by definition ($\boldsymbol{\mathbb{E}[\epsilon] = 0}$), its average interaction with the model's error is perfectly zero.

Regularization

To prevent high-capacity models from overfitting to training data and capturing random noise, machine learning employs a technique known as **regularization**. The core objective function is modified by adding a penalty term that restricts model complexity:

$$\mathcal{L}_{new}(\beta; Z) = \mathcal{L}(\beta; Z) + \lambda \mathcal{R}(\beta)$$

In this formulation, $\mathcal{R}(\beta)$ represents the **regularization term**, which penalizes complex parameter configurations. This term encourages a mathematical preference for simpler or smoother functions within the hypothesis family. Typically, the regularizing function is data-independent, acting as a structural prior belief on the model parameters.

L2 Regularization (Ridge Regression)

One of the most common regularization methods is **L2 Regularization** (also known as **Ridge Regression**), which adds the squared magnitude of the coefficient vector as a penalty:

$$L(\beta; Z) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \beta^{\top} x_i)^2 + \lambda |\beta|_2^2$$

The parameter $\lambda \geq 0$ acts as a hyperparameter that explicitly controls the strength of the regularization.

- **A larger λ** forces greater shrinkage of the coefficients toward zero, yielding a simpler model with reduced variance.
- **A smaller λ** brings the objective closer to ordinary least squares, resulting in lower bias but potentially higher variance.

Geometrically, L2 regularization corresponds to assuming a **Gaussian prior** on the weights. The optimization process finds a balance between fitting the data (minimizing the original Mean Squared Error loss) and keeping the weights small (minimizing the circular L2 penalty). A larger coefficient value is only selected if it provides a significantly better fit for the data.

L1 Regularization (Lasso Regression)

An alternative approach is **L1 Regularization** (commonly referred to as **Lasso Regression**), which penalizes the sum of the absolute values of the coefficients instead of their squares:

$$\mathcal{L}(\beta; Z) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \beta^{\top} x_i)^2 + \lambda |\beta|_1$$

Unlike L2 regularization, which smoothly shrinks all coefficients toward zero without ever reaching it, L1 regularization produces **sparse solutions**. Because the geometric contours of the L1 penalty form sharp corners (resembling a diamond shape), the minimization path frequently intersects these corners directly on the axes. This causes certain parameters (such as β_1) to become exactly zero, effectively performing automatic **feature selection** by removing irrelevant variables from the model entirely.

Tuning Regularization Hyperparameters

Because regularization hyperparameters like λ control the delicate balance between bias and variance, they must be carefully tuned before deployment.

A naive approach involves trying a few candidate values λ_t and selecting the one that minimizes the test loss. However, this method introduces a serious flaw: using the test set to guide hyperparameter selection causes the model to implicitly overfit to the test data, providing an overly optimistic and unrealistic estimate of future performance. This issue worsens exponentially when tuning multiple hyperparameters simultaneously.

To ensure robust evaluation, the recommended procedure follows a structured protocol:

1. **Separate the Dataset:** Divide the available data into three distinct subsets: training, validation, test.
2. **Optimize on Validation:** Train candidate models with various hyperparameter configurations exclusively on the training data, and evaluate their performance on the validation set to select the best model settings.
3. **Final Evaluation:** Assess the chosen model's true generalization capability once on the untouched test set.

This validation-driven framework can be utilized not only for tuning regularization strengths, but also for comparing entirely different model families, feature maps, and learning rates.

Train-Validation-Test Splitting

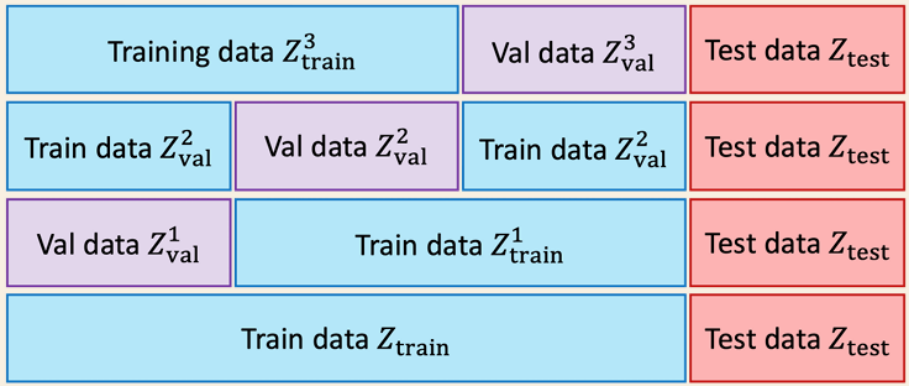
To rigorously evaluate machine learning models and select optimal data is systematically divided into distinct subsets (train, validation, test). A standard rule of thumb is a 60/20/20 split, where the dataset is typically shuffled before being partitioned into a training set (Z_{train}), a validation set (Z_{val}), and a test set (Z_{test}).

The hyperparameter selection process follows a precise protocol:

1. **Split the dataset** into training, validation, and test subsets ($Z \rightarrow Z_{train}, Z_{val}, Z_{test}$).
2. **Evaluate candidates:** For each hyperparameter candidate λ_t (where $t = 1, \dots, h$), train the model on Z_{train} to obtain $\hat{\beta}_{Z_{train},\lambda_t}$ and evaluate its performance on Z_{val} to compute the validation loss $L_{val,t}$. Select the best-performing hyperparameter $t' = \arg\min_t L_{val,t}$ to yield $\lambda_{t'}$
3. **Final training:** Retrain the model on Z_{train} using the selected hyperparameter $\lambda_{t'}$ to produce the final model parameters $\hat{\beta}_{Z_{train},\lambda_{t'}}$

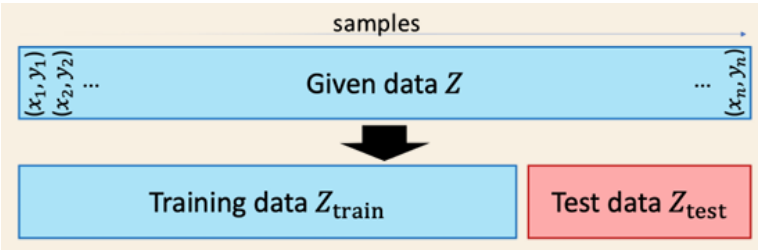
K-Fold Cross Validation

When data is scarce, **K-fold Cross-Validation** is utilized to maximize data usage by iteratively partitioning the training and validation splits across multiple folds.



While increasing the number of folds ($k \rightarrow N$) improves model selection accuracy, it also increases the computational cost.

Detect Overfitting



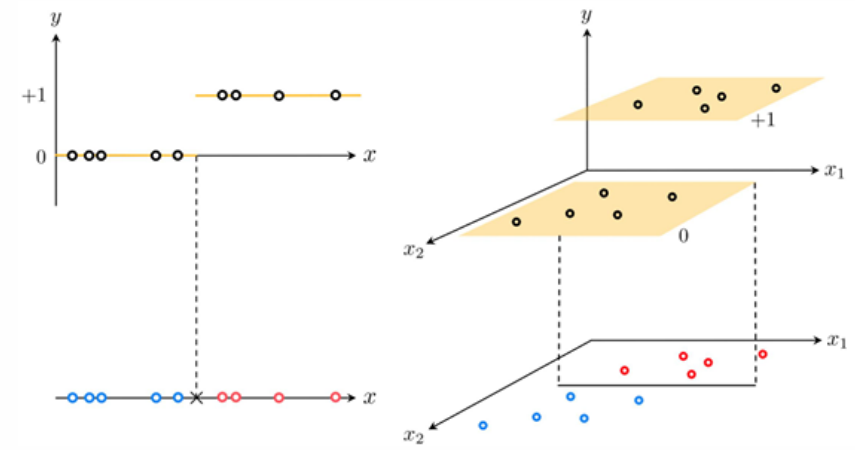
To actively detect overfitting during the training process, data scientists follow a strict validation protocol:

1. **Split the Dataset:** First, shuffle the entire dataset $\boldsymbol{Z}$ and split it into two distinct subsets: a **training set** ($\boldsymbol{Z_{train}}$) and a **test set** ($\boldsymbol{Z_{test}}$).
2. **Fit the Model:** Train the linear model exclusively on the training set to get the optimal parameters $\widehat{\boldsymbol{\beta}}$.
3. **Compare Losses:** Compute the loss (NLL) on the training set and compare it to the loss on the unseen test set. If the training loss is extremely low but the test loss is high, the model has overfitted.

Binary Classification

Until now, we have focused on **regression**, where the goal is to predict a continuous numeric value $y_i \approx f_\beta(x_i)$ (e.g., the price of a house) that belongs to the set of real numbers ($y_i \in \mathbb{R}$), and tries to approximate as much as possible the true value.

However, many real-world problems require **binary classification**, where the goal is to assign an input to one of two distinct categories. In this setting, the labels are strictly discrete, typically represented as $y_i \in \{0, 1\}$.



To adapt a linear model for classification, we do not predict a continuous line; instead, we use a threshold. We predict the class using an **indicator function**, denoted as $\mathbf{1}(C)$, which equals $\mathbf{1}$ if the condition C is true, and $\mathbf{0}$ if it is false. The classification prediction is formulated as $y_i \approx \mathbf{1}(\beta^\top x_i \geq 0)$. To effectively learn the parameters β for this new task, we must define an appropriate loss function.

Maximum Likelihood Estimation (MLE)

To mathematically derive the best loss function for different types of problems, machine learning relies on a probabilistic framework known as **Maximum Likelihood Estimation (MLE)**.

Instead of viewing the model simply as a geometric curve, we assume that for a given input x_i, the target label y_i is drawn independently and identically distributed (i.i.d.) from a specific conditional probability distribution. This distribution is parameterized by our model's weights (β) and is formally written as $y_i \sim p_{Y|X}(y_i|x_i;\beta)$, or more compactly as $p_\beta(y|x)$.

Given this probabilistic model, the **Likelihood** of observing our entire specific training dataset $Z = \{(x_i, y_i)\}_{i=1}^n$ is the product of the individual probabilities for each data point:

$$L(\beta; Z) = \prod_{i=1}^{n} p_\beta(y_i|x_i)$$

Our goal in MLE is to find the parameters β that maximize this Likelihood, meaning we want to find the model that makes our observed data the most mathematically probable.

Negative Log-Likelihood (NLL)

Our goal in MLE is to find the parameters β that maximize this Likelihood, meaning we want to find the model that makes our observed data the most mathematically probable. However, multiplying thousands of tiny probability values together causes severe numerical instability (underflow) in computers.

To solve this, we take the logarithm of the likelihood function, which conveniently transforms the product of probabilities into a sum. Furthermore, because optimization algorithms (like Gradient Descent) are

traditionally designed to minimize rather than maximize, we multiply the result by -1. This gives us the **Negative Log-Likelihood (NLL)**, which serves as a highly effective and numerically stable loss function:

$$l(\beta; Z) = -\sum_{i=1}^{n} \log p_\beta(y_i|x_i).$$

Linear Regression as a Probabilistic Model

The true power of the MLE framework is that it fundamentally explains why we use specific loss functions. We can prove that standard Linear Regression is actually just a specific case of Maximum Likelihood Estimation.

If we assume that the errors in our data follow a normal (Gaussian) distribution, the conditional density of each observation is:

$$p_\beta(y_i|x_i) = \mathcal{N}(y_i; \beta^\top x_i, 1) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}(\beta^\top x_i - y_i)^2\right)$$

When we plug this Gaussian equation into our Negative Log-Likelihood formula, the complex mathematical terms simplify significantly:

$$l(\beta; Z) = \frac{n \log 2\pi}{2} + \frac{1}{2}\sum_{i=1}^{n}(\beta^\top x_i - y_i)^2$$

This resulting equation is simply a mathematical constant added to the **Mean Squared Error (MSE)**. Therefore, minimizing the MSE is mathematically identical to maximizing the probability that a Gaussian distribution generated the data. The model fundamentally predicts the value of y that is most likely given the input x: $f_\beta(x) = \arg\max_y p_\beta(y|x)$. Because the maximum likelihood of a Gaussian distribution occurs exactly at its mean, our optimal prediction perfectly recovers the linear function $f_\beta(x) = \beta^\top x$.

Two Unifying Views of Machine Learning

We can now understand machine learning through two parallel, mathematically equivalent paradigms:

- **The Loss Minimization View:** We define a model family (e.g., linear functions), choose a loss function to measure the "approximation" error (e.g., MSE), and use an optimizer (e.g., Gradient Descent) to minimize that loss.
- **The Maximum Likelihood View:** We define a conditional probability distribution $p_\beta(y|x)$ for our data and use an optimizer to maximize the likelihood (or minimize the NLL) of observing our training dataset.

Logistic Regression

By changing our assumption about the underlying probability distribution, we can adapt our linear model to solve **classification problems rather than regression problems**.

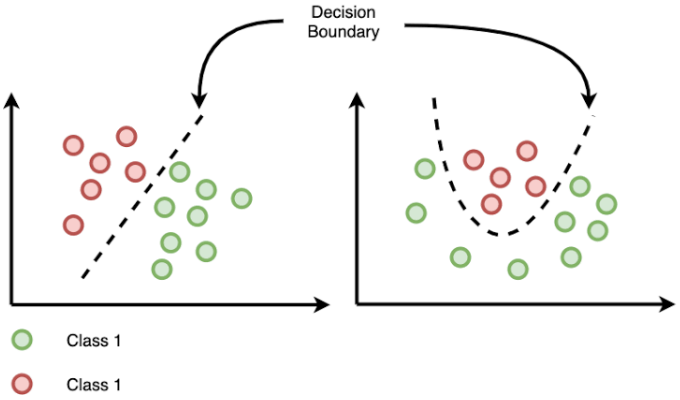
This leads to **Logistic Regression**. In this approach, each data point $x_i \in \mathbb{R}^d$ exists as a coordinate in a multi-dimensional feature space. The linear model computes a raw continuous score, formulated as

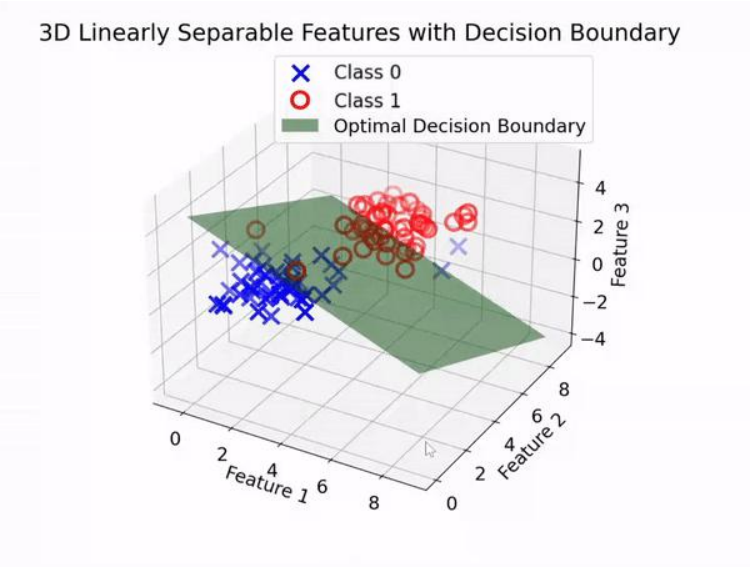
$$s(x_i) = \beta^\top x_i$$

Geometrically, the weights vector β defines a specific vector direction in this feature space, and the scalar score $s(x_i)$ measures exactly how far the input x_i projects along that direction. In upcoming sections, we will explore how this raw score is transformed into a strict probability for binary classification.

The **decision boundary** (the threshold where the model is equally unsure between the two classes) occurs exactly where this raw score equals zero ($\boldsymbol{\beta}^{\top}\boldsymbol{x_i} = \mathbf{0}$). This equation defines a **separating hyperplane** that passes through the origin and is completely perpendicular to the weight vector $\boldsymbol{\beta}$. Consequently, the space is split into two regions:

- points generating a positive score ($\boldsymbol{\beta}^{\top}\boldsymbol{x_i} > \mathbf{0}$) lie on one side and are classified as $\boldsymbol{Y} = \mathbf{1}$
- points generating a negative score ($\boldsymbol{\beta}^{\top}\boldsymbol{x_i} < \mathbf{0}$) lie on the opposite side and are classified as $\boldsymbol{Y} = \mathbf{0}$





The Energy-Based Probabilistic View

To understand how Logistic Regression assigns probabilities, machine learning borrows a concept from statistical physics: the **Boltzmann distribution**. In physics, this distribution states that a system naturally prefers to be in a stable, low-energy state (just like a ball naturally rolls to the bottom of a valley). The lower the energy of a state, the higher the probability that the system will be found there.

Machine learning uses this exact same logic for classification. We treat the model's prediction process as an "energy landscape":

- The model wants to assign a **low energy (high probability)** to the correct class.
- The model wants to assign a **high energy (low probability)** to the incorrect class.

To do this mathematically, the model uses the raw geometric score we calculated earlier ($\boldsymbol{s}(\boldsymbol{x_i}) = \boldsymbol{\beta}^{\top}\boldsymbol{x_i}$). It assigns "energies" to Class 1 and Class 0 using an exponential function.

- If you move further into the positive territory of the weight vector $\boldsymbol{\beta}$, the score $\boldsymbol{\beta}^{\top}\boldsymbol{x_i}$ grows larger. This increases the mathematical likelihood for Class 1 and decreases it for Class 0.
- Therefore, the linear score $\boldsymbol{\beta}^{\top}\boldsymbol{x_i}$ fundamentally acts as a **signed distance** from the boundary line. The further away a point is from the boundary, the more "energy" pushes it toward a specific class.

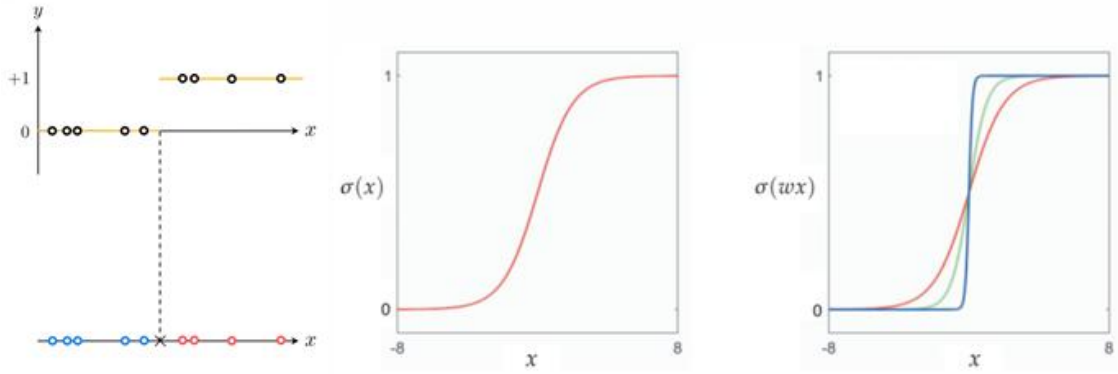
However, while the exponential "energy" values explain the relationship between the two classes, they are just raw numbers. By definition, a true mathematical probability must be a number strictly between $\mathbf{0}$ and $\mathbf{1}$, and the probabilities of all possible classes must sum exactly to $\mathbf{1}$.

To convert our raw exponential energies into true probabilities, we must perform a mathematical step called **normalization** (dividing the values so they sum to 1). After the normalization of the scores, the resulting formula is the **Sigmoid function** (or logistic function):

$$\boldsymbol{p_\beta(Y = 1|x_i) = \frac{1}{1 + e^{-\beta^\top x_i}} = \sigma(\beta^\top x_i)}$$

The Sigmoid function acts as a mathematical "squashing" tool. It takes any raw linear score and smoothly maps it to a strict range between $\mathbf{0}$ and $\mathbf{1}$.

- **The Sign:** If the raw score $\beta^\top x_i$ is positive, the Sigmoid outputs a probability greater than 0.5 (leaning toward Class 1). If it is negative, the probability drops below 0.5 (leaning toward Class 0).
- **The Magnitude:** The absolute size of the score ($|\beta^\top x_i|$) tells us how far the point is from the decision boundary. This represents the model's confidence. A score of $+0.1$ yields a probability of 52% (very uncertain), while a score of $+5.0$ yields a probability of 99% (highly confident).



The Final Logistic Regression Classifier

Ultimately, the purpose of building this probabilistic model is to make a final, concrete decision for new data. The classifier, denoted as $\boldsymbol{f_\beta(x)}$, must output the class that has the highest probability:

$$\boldsymbol{f_\beta(x) = \arg\max_y p_\beta(y|x)}$$

While calculating the exact Sigmoid fraction is necessary during the training phase, making the final prediction is actually much simpler. Because of the mathematical shape of the Sigmoid curve, it only outputs a probability $\geq \mathbf{0.5}$ when the input score is $\geq \mathbf{0}$.

Therefore, to classify a new point, the computer does not need to calculate the complex exponential fraction. It simply looks at the raw geometric score and applies a strict threshold:

$$\boldsymbol{f_\beta(x) = \begin{cases} 1, & \text{if } \beta^\top x \geq 0 \\ 0, & \text{otherwise} \end{cases}}$$

In formal mathematics, this is written using an **indicator function**, $\mathbb{I}$, which outputs $\mathbf{1}$ if the condition is true, and $\mathbf{0}$ if it is false:

$$\boldsymbol{f_\beta(x) = \mathbb{I}(\beta^\top x \geq 0)}$$

This proves that while Logistic Regression is trained using complex probabilities and energy landscapes, its final decision-making process is just a simple check to see which side of the boundary the data point falls on.

Binary Cross-Entropy

For binary Logistic Regression, where $y_i \in \{0,1\}$ and the probability is computed using the sigmoid function $\sigma(\beta^\top x_i)$, the NLL (often called Binary Cross-Entropy) is explicitly formulated as:

$$\boldsymbol{l_\beta(Z) = -\sum_{i=1}^{n}[y_i \cdot \log \sigma\,(\beta^\top x_i) + (1 - y_i) \cdot \log(1 - \sigma(\beta^\top x_i)))]}.$$

This formula dynamically activates different penalty terms based on the true label:

- **Case $y_i = 1$:** The second half of the equation becomes zero. The model is penalized by $-\log \sigma\,(\beta^\top x_i)$. If the model perfectly predicts a probability of 1, the loss is exactly 0. However, if the model confidently predicts 0 for a true 1, the loss approaches infinity (∞), heavily penalizing the model for being confidently wrong.
- **Case $y_i = 0$:** The first half of the equation becomes zero. The model is penalized by $-\log(1 - \sigma(\beta^\top x_i)))$. Similarly, a perfect prediction of 0 yields zero loss, while predicting 1 approaches an infinite penalty.

Gradient of Negative Log-Likelihood

To optimize our Logistic Regression model, we must compute the gradient of the NLL with respect to the parameters $\boldsymbol{\beta}$. By taking the derivative of the loss function and substituting the known derivative of the sigmoid function ($\boldsymbol{\nabla_\beta \sigma = \sigma(1 - \sigma)x_i}$), the complex logarithmic terms cancel out beautifully.

The final, mathematically simplified gradient for binary Logistic Regression is:

$$\boldsymbol{\nabla_\beta l_\beta(Z) = \sum_{i=1}^{n}(\sigma(\beta^\top x_i) - y_i)\, x_i}.$$

Notice that this gradient formulation is structurally identical to the gradient used in standard Linear Regression. Because of this similarity, we can train the classifier using the exact same Gradient Descent algorithm.

However, there is a critical distinction: unlike Linear Regression, setting this gradient to zero does **not** yield a closed-form solution. Because the equations involve non-linear sigmoid functions, we cannot isolate $\boldsymbol{\beta}$ algebraically; we are strictly forced to use iterative optimization techniques like Gradient Descent.

Multi-Class Classification

While binary classification handles only 2 choices (e.g., Spam vs. Not Spam), many problems require categorizing data into multiple distinct classes (e.g., predicting if an image is a cat, dog, or bird). To achieve this, we extend our model to **Multi-class Logistic Regression**.

Instead of using a single parameter vector to draw one decision boundary, the model assigns a separate, unique parameter vector β_y for every individual class label $y \in \mathcal{Y} = \{1, \dots, k\}$. This allows the model to compute an independent raw score for each class. The probability of a data point belonging to a specific class y is defined using exponentiated scores:

$$p_\beta(y|x) = \frac{e^{\beta_y^\top x}}{\sum_{y'} e^{\beta_{y'}^\top x}}$$

This is a valid probability between [0, 1] because exponentiating the raw score ($\beta_y^\top x$), we guarantee that all resulting values are strictly positive. Dividing by the sum of all exponentiated class scores ensures the values are properly normalized, meaning the probabilities across all possible classes sum exactly to 1.

The Softmax Function

This mathematical operation of exponentiating and normalizing is universally known as the **Softmax function**. It is the multi-class generalization of the sigmoid function.

$$\textbf{Softmax}(z_1, \dots, z_k)_i = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$$

By applying the Softmax function to the outputs of our linear equations, we smoothly convert arbitrary geometric projections into a valid, categorical probability distribution. The final probability of a specific class y is simply the Softmax output for that particular class's score relative to all other class scores.

NLL for Multi-Class Classification

To train a multi-class model, we once again apply the Maximum Likelihood Estimation (MLE) framework. For an entire dataset composed of independent and identically distributed (i.i.d.) samples, the total likelihood is the product of the individual Softmax probabilities.

Converting this likelihood into a Negative Log-Likelihood (NLL) loss function yields the following equation (often called Categorical Cross-Entropy):

$$NLL(\boldsymbol{\beta}; \boldsymbol{Z}) = -\sum_{i=1}^{n} \log p\,(Y = y_i|x_i) = -\sum_{i=1}^{n} \left[\boldsymbol{\beta}_{y_i}^\top \boldsymbol{x}_i - \log \sum_{k=1}^{K} \exp(\boldsymbol{\beta}_k^\top \boldsymbol{x}_i)\right].$$

When Gradient Descent minimizes this function, it achieves two things simultaneously:

1. The positive term $\boldsymbol{\beta}_{y_i}^\top \boldsymbol{x}_i$ is maximized, which actively increases the raw score (and thus the likelihood) for the correct, true class.
2. The negative logarithmic term acts as a global normalizer, actively suppressing the scores of all the incorrect alternative classes.

One-Hot Encoding

To mathematically formalize the multi-class Negative Log-Likelihood (NLL) discussed previously, we use a representation called one-hot encoding for our true labels. Instead of a single integer, the label y_i is represented as a **binary vector where only the element corresponding to the true class j is 1**, and all other elements are 0 ($y_{ij} \in \{0,1\}$). Using this notation, the multi-class NLL can be written as:

$$\boldsymbol{NLL(\beta; Z) = -\sum_{i=1}^{n}\sum_{j=1}^{K} y_{ij} \log p\,(Y = j|x_i).}$$

Because $\boldsymbol{y_{ij}}$ is exactly zero for every incorrect class, only the logarithm of the predicted probability for the *true* class contributes to the final loss. This specific formulation is widely known as **cross-entropy loss**.

The optimization objective remains the same: we estimate the optimal parameters $\widehat{\boldsymbol{\beta}}$ by minimizing this NLL. Because of the negative sign, minimizing the NLL is mathematically equivalent to maximizing the original likelihood $\boldsymbol{L(\beta; Z)}$. This loss function behaves very intuitively: it heavily penalizes the model (yielding a high loss) if it assigns a low probability to the correct class, and it rewards the model (yielding a low loss) if it predicts the correct class with high confidence ($\boldsymbol{p(Y = y_i|x_i) \approx 1}$).

Gradient Computation for Multi-Class

To train this model using Gradient Descent, we must compute the gradient of the cross-entropy loss with respect to a specific class's parameter vector, $\boldsymbol{\beta_j}$. The gradient is calculated as:

$$\boldsymbol{\nabla_{\beta_j} NLL(\beta; Z) = -\sum_{i=1}^{n} x_i \left(1_{y_i=j} - p(Y = j|x_i)\right).}$$

This gradient update intuitively guides the learning process: just like in binary classification, it pushes the weights $\boldsymbol{\beta_j}$ to simultaneously increase the predicted probability of the correct class while driving down the probabilities for all incorrect classes.

Classification Metrics

While regression models are evaluated using continuous metrics like MSE, RMSE, and R^2, classification models rely on performance summaries derived from a **confusion matrix**.

A confusion matrix is a table that maps a model's predicted classifications against the true, actual labels. It categorizes predictions into four fundamental outcomes:

- **True Positives (TP):** The model correctly predicts a positive outcome. (Actual=YES, Pred=YES)
- **True Negatives (TN):** The model correctly predicts a negative outcome. (Actual=NO, Pred=NO)
- **False Positives (FP):** The model incorrectly predicts a positive outcome (Actual=NO, Pred=YES)
- **False Negatives (FN):** The model incorrectly predicts a negative outcome (Actual=YES, Pred=NO)

From these foundational counts, several vital performance metrics are derived:

- **Accuracy:** Measures the overall percentage of correct predictions made by the model. It is calculated as $\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$

  However, accuracy can be highly misleading when applied to imbalanced datasets. For example, in a medical test where only 1 out of 100 people has a rare disease, a naive model that always predicts "No Cancer" achieves 99% accuracy, even if it completely fails its primary objective of identifying sick patients.

- **Recall (True Positive Rate):** Measures the model's ability to identify all actual positive cases, defined as Recall $= \frac{TP}{TP+FN}$. Recall is prioritized when False Negatives carry a severe cost, such as in medical screening where missing a real disease is far more dangerous than requesting a follow-up test for a healthy individual.

- **Specificity (True Negative Rate):** Measures the model's ability to avoid false alarms among actual negative cases, defined as Specificity $= \frac{TN}{TN+FP}$. This is essential when False Positives are costly, such as in spam filtering, where blocking a critical work email is worse than letting a minor spam message.
- **Precision:** Evaluates the reliability of positive predictions, calculated as Precision $= \frac{TP}{TP+FP}$

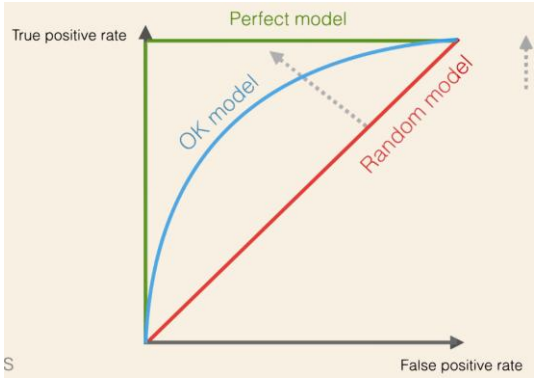
- **F1 Score:** Acts as the harmonic mean of Precision and Recall, computed as $F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

  This is ideal for imbalanced datasets because it balances the need to minimize both False Positives and False Negatives.

Receiver Operating Characteristic (ROC)

Most classification models output a continuous probability (e.g., a 70% chance of spam) rather than a rigid binary decision. Converting these probabilities into final classifications requires setting a **decision threshold** (such as classifying anything above 50% as positive).

The **Receiver Operating Characteristic (ROC) Curve** is a graphical plot that illustrates a model's diagnostic performance across all possible decision thresholds.



The Plot:

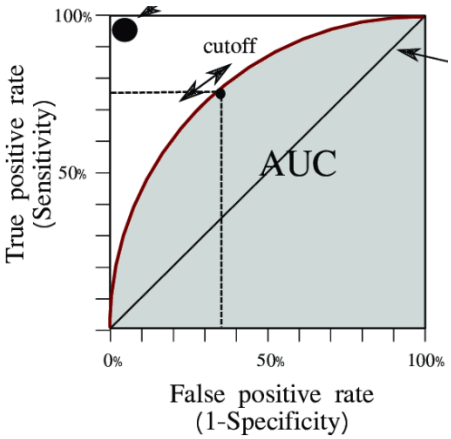
- **Y-axis**: Sensitivity (True Positive Rate)
- X-**axis**: 1 - Specificity (False Positive Rate)

Area Under the ROC Curve (AUC)

To summarize the ROC curve into a single evaluative figure, data scientists use the **Area Under the ROC Curve (AUC)**. The AUC measures the overall capability of the model to correctly separate positive and negative classes regardless of the chosen threshold:

- An AUC $= 1.0$ represents a theoretically perfect model.
- An AUC $= 0.5$ indicates a random, uninformative model.

- An AUC score around 0.8 generally indicates a strong, highly discriminating model.



Metrics Comparison

Metric	Key Question	When to Use It
Accuracy	"What percentage did we get right?"	**Balanced classes**; a simple overview.
Recall	"Of all *actual* positives, how many did we find?"	**Don't miss positives!** (e.g., medical diagnosis)
Specificity	"Of all *actual* negatives, how many did we find?"	**Don't create false alarms!** (e.g., spam filtering)
Precision	"Of all *predicted* positives, how many were real?"	**Don't be wrong when you say 'positive'!** (e.g., YouTube recommendations)
F1-Score	"What's the balance of Precision & Recall?"	**Imbalanced classes**; when you care about both FPs and FNs.
AUC-ROC	"How well can the model separate the classes?"	**Overall model quality**; comparing models, independent of threshold.